

Akademia WSB

WSB University

Kierunek studiów: Informatyka

PRACA DYPLOMOWA INŻYNIERSKA

Imię i nazwisko studenta
Mateusz Kołtowski

Tytuł pracy

Zastosowanie mechanizmu łańcucha bloków do śledzenia newralgicznych zdarzeń
produkcyjnych

Praca inżynierska
napisana pod kierunkiem
dr inż. Paweł Buchwald

SPIS TREŚCI

WSTĘP	4
ROZDZIAŁ I. PODSTAWY TEORETYCZNE	6
1.1. Koncepcja Przemysłu 4.0	6
1.2. Technologia łańcucha bloków jako paradygmat niezaprzeczalności danych.....	7
1.3. Architektura sieci Solana oraz specyfika inteligentnych kontraktów.....	7
1.4. Zdecentralizowane systemy przechowywania danych na przykładzie IPFS	8
1.5. Wykorzystanie mechanizmów kryptograficznych w ochronie danych	9
ROZDZIAŁ II. CHARAKTERYSTYKA PROBLEMU BADAWCZEGO	11
2.1. Sformułowanie problemu badawczego.....	11
2.2. Cel i zakres pracy.....	12
2.3. Założenia projektowe systemu.....	13
2.4. Metody, techniki i narzędzia badawcze	14
ROZDZIAŁ III. PROJEKT KONCEPCYJNY SYSTEMU.....	16
3.1. Architektura i główne założenia systemu	16
3.2. Dwutorowa strategia rejestracji zdarzeń.....	19
3.3. Zastosowane mechanizmy kryptograficzne i procedura audytu.....	21
ROZDZIAŁ IV. IMPLEMENTACJA SYSTEMU.....	23
4.1. Symulator urzędów brzegowych.....	23
4.2. Brama sieciowa.....	27
4.3. Inteligentny kontrakt w sieci Solana.....	32
4.4. Aplikacja audytorska i weryfikacja danych.....	34
4.5. Panel operatorski WebUI.....	36
ROZDZIAŁ V. TESTY I ANALIZA WYNIKÓW.....	40
5.1. Środowisko testowe i konfiguracja infrastruktury	40
5.2. Weryfikacja funkcjonalna systemu.....	42
5.2.1. Rejestracja anomalii w czasie rzeczywistym	42
5.2.2. Cykliczna archiwizacja danych i integracja z IPFS.....	43
5.2.3. Wymiana danych za pomocą protokołu Modbus TCP	44
5.2.4. Wizualizacja danych w czasie rzeczywistym	45
5.3. Ocena procedury audytu i odporności na sabotaż	46

5.4. Analiza ekonomiczno-energetyczna	50
5.4.1. Bilans kosztów transakcyjnych w sieci Solana	50
5.4.2. Efektywność energetyczna architektury	51
5.5. Ograniczenia i słabe strony zaproponowanego rozwiązania	52
ZAKOŃCZENIE	54
BIBLIOGRAFIA	55
WYKAZ TABEL	56
WYKAZ RYSUNKÓW	57

WSTĘP

Współczesne systemy sterowania produkcją opierają się na architekturach, w których dane z czujników trafiają do scentralizowanych, lokalnych baz danych. Choć takie rozwiązanie sprawdza się w codziennej eksploatacji, stanowi istotny punkt podatności. Rejestry zapisane w ten sposób narażone są na nieautoryzowaną modyfikację lub trwałe usunięcie, co podważa wiarygodność systemu audytowego. Problem ten nabiera szczególnego znaczenia w sektorach poddanych ścisłej regulacji prawnej, takich jak przemysł farmaceutyczny czy motoryzacyjny, gdzie wymagana jest pełna niezaprzeczalność zdarzeń produkcyjnych.

Niniejsza praca odpowiada na ten problem, proponując projekt i implementację systemu opartego na technologiach zdecentralizowanych. Celem jest zapewnienie niezaprzeczalności i integralności logów z newralgicznych zdarzeń produkcyjnych bez konieczności ufania administratorowi lokalnej bazy danych. Wykorzystano w tym celu publiczny łańcuch bloków Solana oraz rozproszony system plików IPFS, a poszczególne moduły zaimplementowano w językach Rust i Python. Opracowane rozwiązanie otrzymało nazwę MateChain.

Zakres pracy obejmuje:

- analizę problemu rejestracji zdarzeń produkcyjnych w istniejących systemach SCADA,
- przegląd technologii łańcucha bloków, ze szczególnym uwzględnieniem platformy Solana oraz rozproszonego systemu plików IPFS,
- projekt koncepcyjny architektury dwutorowej rejestracji zdarzeń,
- implementację symulatorów urządzeń produkcyjnych, bramy integracyjnej, inteligentnego kontraktu oraz panelu wizualizacyjnego,
- testy funkcjonalne weryfikujące poprawność działania systemu,
- opracowanie procedury audytu umożliwiającej odzyskanie i weryfikację archiwizowanych danych.

Praca obejmuje pełny cykl prac inżynierskich. W pierwszym rozdziale przedstawiono podstawy teoretyczne i przegląd istniejących rozwiązań. Drugi rozdział zawiera charakterystykę problemu badawczego, a trzeci - koncepcję i architekturę proponowanego rozwiązania. Rozdział czwarty omawia szczegóły implementacji poszczególnych

modułów systemu MateChain. Pracę zamykają testy, analiza wyników oraz wnioski końcowe.

Pełny kod źródłowy systemu MateChain, stanowiącego praktyczną realizację założeń niniejszej pracy, dołączono jako załącznik elektroniczny do systemu uczelnianego oraz udostępniono w publicznym repozytorium platformy GitHub https://github.com/mkoltowski/mate_chain.

ROZDZIAŁ I. PODSTAWY TEORETYCZNE

1.1. Koncepcja Przemysłu 4.0

Przemysł 4.0 to koncepcja, w której urządzenia i systemy technologiczne komunikują się ze sobą, a duże ilości danych produkcyjnych poddawane są analizie. Wiąże się z nią zmiana architektury systemów zarządzania produkcją¹.

Blisko powiązane z Przemysłem 4.0 jest pojęcie „inteligentnej fabryki”. Określa się tak zakład oparty na systemach cyber-fizycznych, zintegrowanych z przemysłowym Internetem Rzeczy, co ma umożliwić prowadzenie procesów wytwórczych przy ograniczonym udziale pracowników. Przykładem jest sieć czujników monitorujących zużycie energii przez maszyny i instalacje technologiczne; pozyskane dane mogą następnie zostać przeanalizowane przez dedykowane oprogramowanie².

Choć koncepcja ta daje dostęp do dużej ilości danych przydatnych w optymalizacji kosztów, jej wdrożenie tworzy także nowe zagrożenia w obszarze cyberbezpieczeństwa. Przedsiębiorstwa coraz częściej dostrzegają ryzyko wynikające m.in. z niewłaściwego przechowywania danych³.

Najczęstszym typem luk w zabezpieczeniach komponentów są przepełnienia bufora. Jest to błąd programistyczny polegający na tym, że podczas zapisu danych do bufora oprogramowanie przekracza jego granice i nadpisuje sąsiednie komórki pamięci. Skutkiem może być utrata danych, awaria programu lub uruchomienie złośliwego oprogramowania⁴.

Krytycznym celem ataku bywają również scentralizowane bazy danych, powszechnie stosowane w przemyśle. Zagrożenie stanowi nie tylko zewnętrzny napastnik, lecz także nieuczciwy pracownik, który może celowo sfalszować logi produkcyjne, a

¹ C. Mychlewicz, Z. Piątek (red.), *Od Industry 4.0 do Smart Factory. Poradnik menedżera i inżyniera*, Warszawa 2017, s. 5.

² Tamże, s. 6.

³ R. Siudak, *Cyberbezpieczeństwo przemysłu – polskie szanse, inwestycje, innowacje*, „Brief Programowy” 2020, nr lipiec, s. 1.

⁴ O. Andreeva, S. Gordeychik, G. Gritsai i in., *Industrial Control Systems Vulnerabilities Statistics*, Kaspersky Lab, 2015, s. 14.

nawet zdarzenia losowe, takie jak utrata serwera w wyniku pożaru. Wymusza to stosowanie metod zabezpieczeń opartych na zapisie danych w miejscach odpornych na tego typu incydenty⁵.

1.2. Technologia łańcucha bloków jako paradygmat niezaprzeczalności danych

Odpowiedzią na opisane luki i ograniczenia scentralizowanych baz danych jest technologia łańcucha bloków w której informacje grupowane są w bloki łączone ze sobą za pomocą jednokierunkowych funkcji kryptograficznych. Mechanizm ten ma zapewnić integralność i niezmiennność historycznych zdarzeń⁶.

Bezpieczeństwo tego rozwiązania opiera się na powiązaniu kolejnych partii danych w nierozdzielalną sekwencję. Każdy nowo dodawany blok zawiera kryptograficzny skrót (hash) bloku poprzedniego, dzięki czemu ingerencja w zapis historyczny zmienia jego skrót i wymusza modyfikację całej dalszej części łańcucha, co ujawnia próbę fałszerstwa pozostałym uczestnikom sieci.

W kontekście systemów przemysłowych istotną cechą łańcucha bloków jest brak pojedynczego punktu awarii. Sieć jest replikowana pomiędzy wieloma niezależnymi węzłami walidującymi, dzięki czemu atak na pojedyncze urządzenie lub jego fizyczne uszkodzenie nie zaburza działania całości. Każda transakcja wymaga przy tym podpisu cyfrowego nadawcy, realizowanego za pomocą kryptografii asymetrycznej opartej na parach kluczy. Połączenie replikacji danych z weryfikacją podpisów pozwala traktować sieci zdecentralizowane jako środowisko odpowiednie do realizacji niezaprzeczalności zdarzeń w Przemysle 4.0.

1.3. Architektura sieci Solana oraz specyfika inteligentnych kontraktów

Dobór infrastruktury rozproszonego rejestru dla systemów przemysłowych wymaga uwzględnienia takich parametrów, jak wysoka przepustowość, niskie opóźnienia i minimalne koszty transakcyjne. Wymagania te spełnia sieć Solana, która w odróżnieniu

⁵ R. Siudak, dz. cyt., s. 1

⁶ A. Yakovenko, Solana: A new architecture for a high performance blockchain v0.8.13, [b.m.w.] 2018, s. 2.

od klasycznych łańcuchów bloków wprowadza mechanizm dowodu historii⁷. Pozwala on szybko finalizować transakcje i osiągać teoretyczną przepustowość rzędu 710 tys. operacji na sekundę, co czyni tę sieć dogodnym środowiskiem wymiany danych w Przemśle 4.0⁸.

Tworzenie oprogramowania w sieci Solana różni się od innych łańcuchów bloków. Inteligentne kontrakty (nazywane tu programami) nie przechowują danych, a jedynie wykonują polecenia; wszystkie informacje zapisywane są w osobnych, powiązanych kontach. Taki podział umożliwia równoległe wykonywanie wielu transakcji, wymaga jednak od programisty rygorystycznej walidacji danych wejściowych i samodzielnego zarządzania pamięcią⁹.

Tworzenie bezpiecznych kontraktów ułatwia framework Anchor. Przejmuje on wiele złożonych zadań, takich jak przydzielanie kont, i chroni kod przed typowymi błędami¹⁰. Generuje też plik IDL czyli ustandaryzowany opis programu w formacie JSON, pełniący funkcję jego instrukcji obsługi. Ułatwia to integrację łańcucha bloków z urządzeniami zewnętrznymi, takimi jak przemysłowe bramy sieciowe czy systemy SCADA¹¹.

1.4. Zdecentralizowane systemy przechowywania danych na przykładzie IPFS

Zapisywanie dużych ilości danych bezpośrednio w łańcuchu bloków, takim jak Solana, jest kosztowne i obciąża sieć. Aby ograniczyć koszty, stosuje się rozproszone systemy plików. Najpopularniejszym z nich jest protokół IPFS, umożliwiający tanie przechowywanie plików w sieci urządzeń połączonych bezpośrednio ze sobą (peer-to-peer)¹². W klasycznej architekturze zasoby identyfikuje się przez fizyczne adresy serwerów natomiast IPFS odwraca tę logikę i wprowadza adresowanie oparte na treści. System odwołuje się nie do lokalizacji pliku, lecz do jego kryptograficznego skrótu (CID), wygenerowanego z samej zawartości danych¹³.

⁷ Tamże, s. 3.

⁸ Tamże, s. 1.

⁹ Źródło: Accounts Solana, www.solana.com/docs/core/accounts z dnia 4.06.2026 r.

¹⁰ Źródło: Anchor framework, www.anchor-lang.com z dnia 4.06.2026 r.

¹¹ Źródło: IDLs (Interface Definition Language) | Solana, www.solana.com/docs/core/idl z dnia 4.06.2026 r.

¹² J. Benet, IPFS - Content Addressed, Versioned, P2P File System, [b.m.w.] 2014, s. 1.

¹³ Tamże, s. 2.

W sieci IPFS każdy udostępniany plik jest dzielony na mniejsze fragmenty (bloki), a z każdego z nich wyliczany jest skrót kryptograficzny. Dzięki temu identyfikator CID jest powiązany z rzeczywistą zawartością pliku a każda próba modyfikacji danych zmienia ten skrót i może zostać wykryta jako ingerencja¹⁴. Taka architektura sprzyja zachowaniu wiarygodnej historii działań w przemyśle.

Rozwiązania przemysłowe oparte na IPFS pozwalają rozdzielić informacje na dwa strumienie. Obszerne, zaszyfrowane dane produkcyjne przechowywane są w sieci rozproszonej (ang. off-chain), natomiast do niezmiennego łańcucha bloków (ang. on-chain) trafia jedynie identyfikator CID, stanowiący trwały dowód ich autentyczności. Rozproszone tablice mieszające umożliwiają szybkie odnalezienie pliku z logami maszyn pracujących na hali, co tworzy obieg danych nieobciążający głównej sieci. To podejście hybrydowe ogranicza problem pojedynczego punktu awarii i stanowi podstawę systemu MateChain.

1.5. Wykorzystanie mechanizmów kryptograficznych w ochronie danych

Podstawą bezpieczeństwa nowoczesnych systemów przemysłowych i sieci rozproszonych jest zastosowanie zaawansowanych mechanizmów kryptograficznych. Ich celem jest realizacja trzech filarów cyberbezpieczeństwa: poufności, integralności i autentyczności przesyłanych informacji. Ponieważ w Przemśle 4.0 dane telemetryczne często stanowią tajemnicę przedsiębiorstwa, stosuje się w tym celu kombinację kilku niezależnych algorytmów.

Poufność danych zapewnia się zwykle za pomocą kryptografii symetrycznej, która wykorzystuje ten sam tajny klucz do szyfrowania i deszyfrowania. Standardem w tej dziedzinie jest algorytm AES. Dzięki operowaniu na blokach danych i wielorundowym transformacjom matematycznym zapewnia on wysoką odporność na ataki siłowe, co pozwala bezpiecznie przysłać wrażliwe logi produkcyjne przez publiczne sieci teleinformatyczne¹⁵.

¹⁴ Źródło: How IPFS works | IPFS Docs, <https://docs.ipfs.tech/concepts/how-ipfs-works/#subsystems-overview> z dnia 4.06.2026 r.

¹⁵ Advanced Encryption Standard (AES), FIPS PUB 197, National Institute of Standards and Technology, Gaithersburg 2001, s. 1–5

Drugim elementem systemów niezaprzeczalności są jednokierunkowe funkcje skrótu, takie jak SHA-256. Przekształcają one dane wejściowe o dowolnej długości w ciąg znaków o stałej długości i cechują się dużą czułością na zmiany najmniejsza modyfikacja danych wejściowych powoduje powstanie zupełnie innego skrótu. Funkcje te stanowią podstawę weryfikacji integralności plików w sieciach rozproszonych typu IPFS¹⁶.

Trzecim elementem architektury bezpieczeństwa jest kryptografia asymetryczna, wykorzystywana do tworzenia podpisów cyfrowych. W odróżnieniu od metod symetrycznych opiera się ona na powiązanej matematycznie parze kluczy: klucz prywatny zna wyłącznie nadawca, a klucz publiczny jest ogólnodostępny. Schematy oparte na krzywych eliptycznych (w tym algorytm stosowany natywnie w sieci Solana) pozwalają urządzeniom brzegowym generować unikalny podpis dla każdej transakcji, dzięki czemu odbiorca może jednoznacznie zweryfikować tożsamość nadawcy. Skuteczność tych mechanizmów zależy jednak od bezpiecznego środowiska programistycznego. W projekcie wykorzystano język Rust, którego rygorystyczne zarządzanie pamięcią eliminuje błędy krytyczne (np. przepełnienia bufora) mogące prowadzić do kompromitacji kluczy¹⁷.

¹⁶ *Secure Hash Standard (SHS)*, FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg 2015, s. 3–4

¹⁷ S. Klabnik, C. Nichols, *Programowanie w Języku Rust Oficjalny Podręcznik Wydanie II*, Gliwice 2023, s. 91.

ROZDZIAŁ II. CHARAKTERYSTYKA PROBLEMU BADAWCZEGO

2.1. Sformułowanie problemu badawczego

Rozwój koncepcji Przemysłu 4.0 i integracja warstwy informatycznej z technologiami operacyjnymi przynoszą wymierne korzyści, ale jednocześnie tworzą nowe wyzwania w obszarze cyberbezpieczeństwa. Tradycyjne systemy sterowania produkcją klasy SCADA, projektowane pierwotnie jako sieci odizolowane, przestały być zamkniętymi środowiskami. Po otwarciu na komunikację sieciową stają się równie podatne na ataki co klasyczne systemy biznesowe, a ich przestarzałe architektury często nie mają wbudowanych mechanizmów odporności na zagrożenia zewnętrzne¹⁸.

Głównym problemem badawczym i inżynierskim niniejszej pracy jest brak mechanizmów gwarantujących niezaprzeczalność i integralność logów produkcyjnych w klasycznych, scentralizowanych bazach danych. Powszechnie stosowane urządzenia brzegowe i komponenty automatyki obciążone są licznymi lukami (omówionymi w poprzednim rozdziale), przez co stają się łatwym celem ataków zarówno zewnętrznych cyberprzestępców, jak i nieuczciwych pracowników zakładu¹⁹. Przełamanie zabezpieczeń daje atakującemu dostęp do głównego serwera zarządzania, który w tradycyjnym modelu opiera się wyłącznie na scentralizowanym zaufaniu. Umożliwia to nieautoryzowaną modyfikację lub usunięcie historii zdarzeń, w wyniku czego zakładowy system monitoringu traci wartość audytorską i dowodową²⁰.

W razie awarii, takiej jak krytyczne przekroczenie temperatury procesu wtryskarki, zapisy w scentralizowanej bazie relacyjnej mogą zostać celowo sfalszowane, aby ukryć błędy ludzkie lub wady technologiczne. Tradycyjne dzienniki zdarzeń nie dostarczają kryptograficznego dowodu na to, że ich stan nie uległ manipulacji od chwili wygenerowania pomiaru przez maszynę. Zabezpieczenia takich systemów opierają się wyłącznie na zaufaniu do administratora sieci i fizycznym ograniczaniu dostępu. W dobie mgły

¹⁸ C. Mychlewicz, Z. Piątek (red.), dz. cyt., s. 26.

¹⁹ O. Andreeva, S. Gordeychik, G. Gritsai i in., dz. cyt., s. 18.

²⁰ Tamże, s. 14.

obliczeniowej i Przemysłowego Internetu Rzeczy, gdzie granice sieci firmowej ulegają zatarciu, takie podejście jest niewystarczające²¹.

Głównym wyzwaniem inżynierskim jest zatem zbudowanie systemu, który zastępuje scentralizowane zaufanie kryptograficzną niezaprzeczalnością i chroni zapisy o awariach maszyn przed sabotażem. Wymagana jest przy tym architektura zapewniająca skalowalność ekonomiczną oraz zgodność ze standardami komunikacyjnymi już funkcjonującymi na hali produkcyjnej.

2.2. Cel i zakres pracy

Głównym celem inżynierskim pracy jest zaprojektowanie, implementacja i weryfikacja funkcjonalna autorskiego systemu MateChain, stanowiącego odpowiedź na zdiagnozowane problemy bezpieczeństwa. System ma zapewnić niezaprzeczalność i integralność logów z newralgicznych zdarzeń produkcyjnych, wykorzystując łańcuch bloków oraz zdecentralizowany system plików ograniczający koszty. Celem aplikacyjnym jest stworzenie bezpiecznego i skalowalnego ekonomicznie rozwiązania, które zastępuje zaufanie do administratora lokalnej bazy danych dowodami kryptograficznymi, zachowując zgodność ze standardami Przemysłu 4.0.

Dla osiągnięcia tego celu zdefiniowano zakres prac badawczo-rozwojowych obejmujący:

- analizę problemu bezpieczeństwa w klasycznych systemach nadzoru przemysłowego oraz przegląd możliwości technologii blockchain (ze szczególnym uwzględnieniem sieci Solana) i rozproszonego systemu plików IPFS,
- opracowanie koncepcyjnego projektu systemu wprowadzającego dwutorową architekturę rejestracji zdarzeń (strumień natychmiastowych alarmów oraz strumień cyklicznej archiwizacji),
- implementację warstwy brzegowej w postaci symulatorów urządzeń produkcyjnych (wtryskarek), napisanych w języku Python i wyizolowanych w środowisku kontenerowym Podman,

²¹ R. Siudak, dz. cyt., s. 1.

- zaprogramowanie wielowątkowej bramy sieciowej w języku Rust, odpowiedzialnej za komunikację, klasyfikację zdarzeń, obsługę protokołu Modbus TCP oraz serwowanie panelu WebUI,
- wdrożenie dedykowanego inteligentnego kontraktu w publicznej sieci Solana z wykorzystaniem frameworka Anchor,
- stworzenie niezależnej aplikacji audytorskiej, umożliwiającej zewnętrznemu kontrolerowi pobranie, weryfikację kryptograficzną i deszyfrację archiwalnych paczek danych z pominięciem zakładowej bazy danych,
- przeprowadzenie testów funkcjonalnych i wydajnościowych weryfikujących poprawność oraz odporność infrastruktury na próby sabotażu.

2.3. Założenia projektowe systemu

Projekt systemu MateChain oparto na założeniach funkcjonalnych i нефункциональных, ujętych w pięciu filarach, które zdeterminowały wybór technologii oraz logikę działania poszczególnych modułów. Do głównych założeń projektowych należą:

Niezaprzeczalność zdarzeń: zdarzenia krytyczne, takie jak gwałtowne przekroczenia dopuszczalnych parametrów technologicznych czy awarie maszyn, muszą być rejestrowane nieodwracalnie, bez możliwości późniejszej modyfikacji lub usunięcia przez personel zakładu. Realizuje to niezmienny, rozproszony rejestr publicznej sieci łańcucha bloków (Solana).

Skalowalność ekonomiczna: maszyny przemysłowe nieustannie wysyłają nowe dane, więc zapisywanie każdego pomiaru w sieci Solana byłoby zbyt kosztowne. Zastosowano model hybrydowy: surowe logi trafiają do rozproszonego systemu plików IPFS, pełniącego rolę taniego magazynu zewnętrznego, natomiast do łańcucha bloków wysyłany jest jedynie identyfikator CID oraz skrót SHA-256, potwierdzający, że plik nie został podmieniony. Mechanizm ten łączy wysoki poziom bezpieczeństwa z niskim kosztem.

Reakcja w czasie rzeczywistym: w przemyśle spóźniona informacja o incydencie może mieć poważne skutki, dlatego architektura nadaje najwyższy priorytet alarmom krytycznym. Są one przesyłane do inteligentnego kontraktu w sieci Solana bez kolejkowania, co zapewnia reakcję i finalizację zapisu w ułamku sekundy.

Poufność danych przemysłowych: aby chronić tajemnicę przedsiębiorstwa, zastosowano dwa poziomy szyfrowania AES-256. Tryb CBC zabezpiecza surowe pomiary przesyłane z maszyn do bramy, a tryb GCM chroni duże archiwa wysyłane do publicznej sieci IPFS. Jednocześnie brama celowo udostępnia dane lokalne bez szyfrowania, tworząc interfejs (Modbus i WebUI) zgodny z dotychczasowym oprogramowaniem SCADA.

Wsteczna kompatybilność i niezależność systemowa: wdrożenie nie wymaga kosztownej przebudowy sprzętowej zakładu, a architektura jest niezależna od konkretnych dostawców sprzętu (tzw. vendor lock-in). System opiera się na otwartym oprogramowaniu i pełni rolę uniwersalnej bramki współpracującej z tradycyjnymi systemami SCADA dzięki obsłudze standardowego protokołu Modbus TCP, od lat stanowiącego fundament automatyki przemysłowej.

2.4. Metody, techniki i narzędzia badawcze

Projektowanie systemu MateChain wymagało zdefiniowania warsztatu inżynierskiego. Zgodnie ze standardami akademickimi metody badawcze to powtarzalne sposoby analizy danych, służące do znalezienia optymalnych odpowiedzi na postawiony problem badawczy²². techniki to bardziej szczegółowe czynności pozwalające gromadzić i przechowywać niezbędne informacje.

Główną metodą zastosowaną w pracy jest metoda eksperymentalna, polegająca na autorskim wdrożeniu i weryfikacji dwutorowej architektury. Wsparto ją metodą symulacyjną, która posłużyła do stworzenia wirtualnych wtryskarek generujących syntetyczne anomalie bez ingerencji w fizyczną fabrykę. Techniki badawcze objęły analizę logów, weryfikację kryptograficzną oraz testy wydajnościowe, zrealizowane za pomocą niezależnego skryptu audytorskiego.

Do zbudowania zintegrowanego systemu wykorzystano następujące narzędzia:
– Język Rust to wysokowydajny, bezpieczny pamięciowo język programowania; posłużył do napisania wielowątkowej bramy sieciowej (gateway), serwera WebUI oraz obsługi protokołu Modbus TCP.

²² Standard pisania prac inżynierskich, Akademia WSB, Dąbrowa Górnicza [b.r.w.], s. 2.

- Język Python to dynamiczny język skryptowy; użyto go do szybkiego prototypowania symulatorów urządzeń brzegowych oraz do stworzenia niezależnego oprogramowania audytora.
- Baza danych SQLite to lekki, relacyjny silnik bazy danych zastosowany jako lokalny bufor off-chain, umożliwiający bezpieczne i współbieżne gromadzenie odczytów przed ich archiwizacją.
- Framework Anchor to narzędzie programistyczne wykorzystane do bezpiecznego i zoptymalizowanego wdrożenia inteligentnego kontraktu²³.
- Solana (Devnet) to publiczna sieć rozproszonych rejestrów, pełniąca rolę kotwicy zaufania dla transakcji on-chain i zapewniająca niezaprzeczalność zdarzeń krytycznych w czasie rzeczywistym²⁴.
- IPFS (Kubo) to oficjalny węzeł rozproszonego systemu plików peer-to-peer, wykorzystany do zdecentralizowanego archiwizowania zaszyfrowanych paczek off-chain²⁵.
- Podman to środowisko wirtualizacji kontenerowej zastosowane do izolacji symulatorów maszyn brzegowych i zapewnienia powtarzalności infrastruktury testowej.

²³ *Anchor framework, dz. cyt.*

²⁴ *A. Yakovenko, dz. cyt., s. 1.*

²⁵ *J. Benet, dz. cyt., s. 1.*

ROZDZIAŁ III. PROJEKT KONCEPCYJNY SYSTEMU

Celem rozdziału jest przedstawienie autorskiej koncepcji i architektury systemu MateChain, stanowiącego odpowiedź na zdefiniowane wcześniej problemy bezpieczeństwa scentralizowanych sieci przemysłowych. Omówiono w nim logiczny podział na warstwy, założenia dotyczące przepływu danych oraz dwutorową strategię rejestracji zdarzeń, wykorzystującą publiczny łańcuch bloków Solana i rozproszony system plików IPFS.

3.1. Architektura i główne założenia systemu

Projektując system monitorowania i rejestracji logów produkcyjnych dla Przemysłu 4.0, należało uwzględnić specyficzne wymagania tego sektora: konieczność zachowania ciągłości pracy maszyn, kompatybilność ze starszymi standardami przemysłowymi oraz odporność na manipulacje danymi historycznymi. Aby sprostać tym wymaganiom, architekturę systemu MateChain oparto na modelu trójwarstwowym.

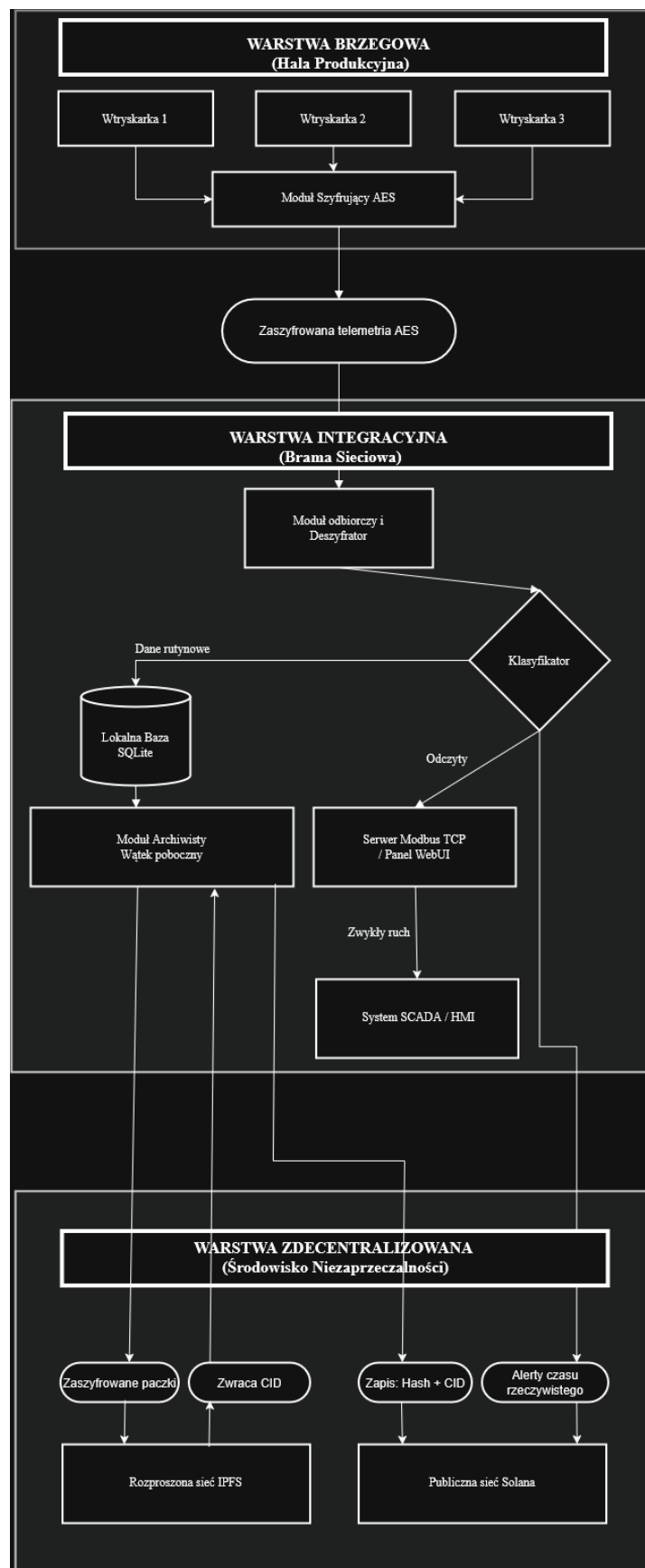
Pierwszą warstwę stanowi środowisko brzegowe, w którym pracują urządzenia produkcyjne. W projekcie reprezentują je symulatory wtryskarek, generujące w czasie rzeczywistym dane telemetryczne (np. temperaturę procesu) z uwzględnieniem zakłóceń i losowych awarii. Warstwa ta wstępnie zabezpiecza przesyłane ładunki kryptografią symetryczną, co chroni ruch sieciowy przed nasłuchem wewnątrz hali produkcyjnej.

Drugim, centralnym elementem architektury jest warstwa integracyjna, realizowana przez autorską bramę sieciową. Pełni ona rolę bufora, tłumacza protokołów i klasyfikatora zdarzeń. Brama odbiera zaszyfrowane komunikaty z warstwy brzegowej, a jednocześnie udostępnia znormalizowane dane systemom klasy SCADA przez standardowy interfejs Modbus TCP. Jej kluczowym zadaniem jest natychmiastowa ocena napływających pomiarów i rozstrzygnięcie, czy dany odczyt jest wartością rutynową, czy krytyczną anomalią technologiczną.

Ostatnią warstwę tworzy zdecentralizowane środowisko niezaprzeczalności, obejmujące współpracę bramy z publiczną siecią Solana oraz węzłem IPFS. Jej zadaniem jest

trwale, kryptograficzne wiązanie historii maszyn z niezmiennym rejestrem, co eliminuje konieczność ufania administratorowi lokalnej bazy SQLite.

Połączenie omówionych trzech warstw tworzy rozwiązanie hybrydowe, które z sukcesem wprowadza innowacje Web3 do hermetycznego świata przemysłu bez ingerencji w dotychczasową infrastrukturę sprzętową. Topologię systemu MateChain przedstawiono na poniższym schemacie.



Rysunek 1 Trójwarstwowa architektura systemu

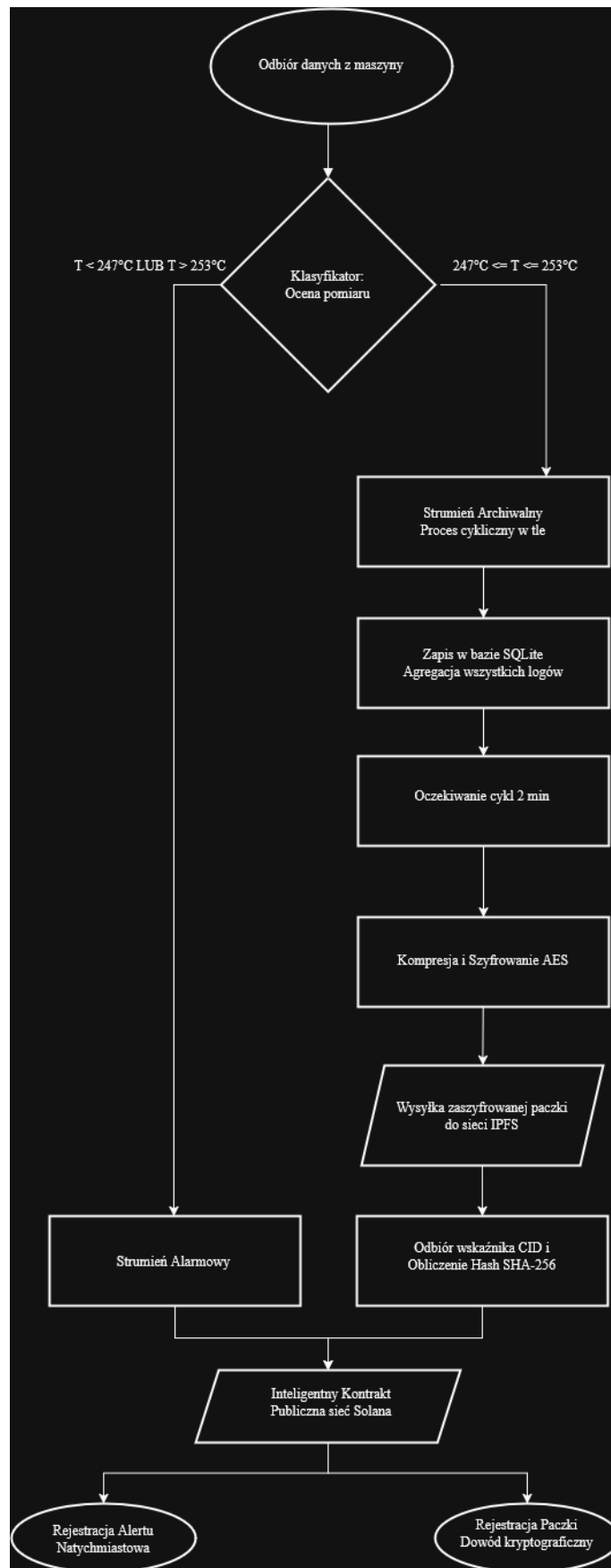
Źródło: opracowanie własne

3.2. Dwutorowa strategia rejestracji zdarzeń

Istotnym problemem przy integracji systemów przemysłowych z technologią blockchain są koszty i ograniczona przepustowość. Zapisywanie każdego odczytu bezpośrednio w łańcuchu bloków generowałoby wysokie opłaty i mogłoby obciążyć sieć. Z drugiej strony całkowita rezygnacja z takiego zapisu pozbawiłaby system jego głównej zalety, czyli gwarancji niezmienności danych. Aby pogodzić te wymagania, w systemie MateChain zastosowano strategię dwutorową, dzielącą napływające informacje na dwa strumienie o różnym priorytecie.

Pierwszy z nich to strumień alarmowy, służący do natychmiastowego zgłaszania poważnych awarii. Gdy brama wykryje, że temperatura spadła poniżej 247°C lub przekroczyła 253°C, system od razu wysyła komunikat do inteligentnego kontraktu w sieci Solana. Powstaje w ten sposób trwały zapis, którego pracownik fabryki nie może później zmodyfikować ani usunąć.

Drugi to strumień archiwalny, działający w tle i agregujący wszystkie standardowe zdarzenia. Niezależnie od stanu maszyny system najpierw zapisuje odczyty w lokalnej bazie SQLite. Następnie dedykowany proces cyklicznie kompresuje te dane, szyfruje je i wysyła do sieci IPFS, która zwraca unikalny identyfikator pliku (CID). Identyfikator ten wraz ze skrótem SHA-256 paczki trafia ostatecznie do rejestru Solana. Podział na dwa niezależne strumienie przetwarzania oraz sekwencję działań bramy zilustrowano na poniższym schemacie.



Rysunek 2 Schemat blokowy dwutorowej strategii rejestracji zdarzeń

Źródło: opracowanie własne

3.3. Zastosowane mechanizmy kryptograficzne i procedura audytu

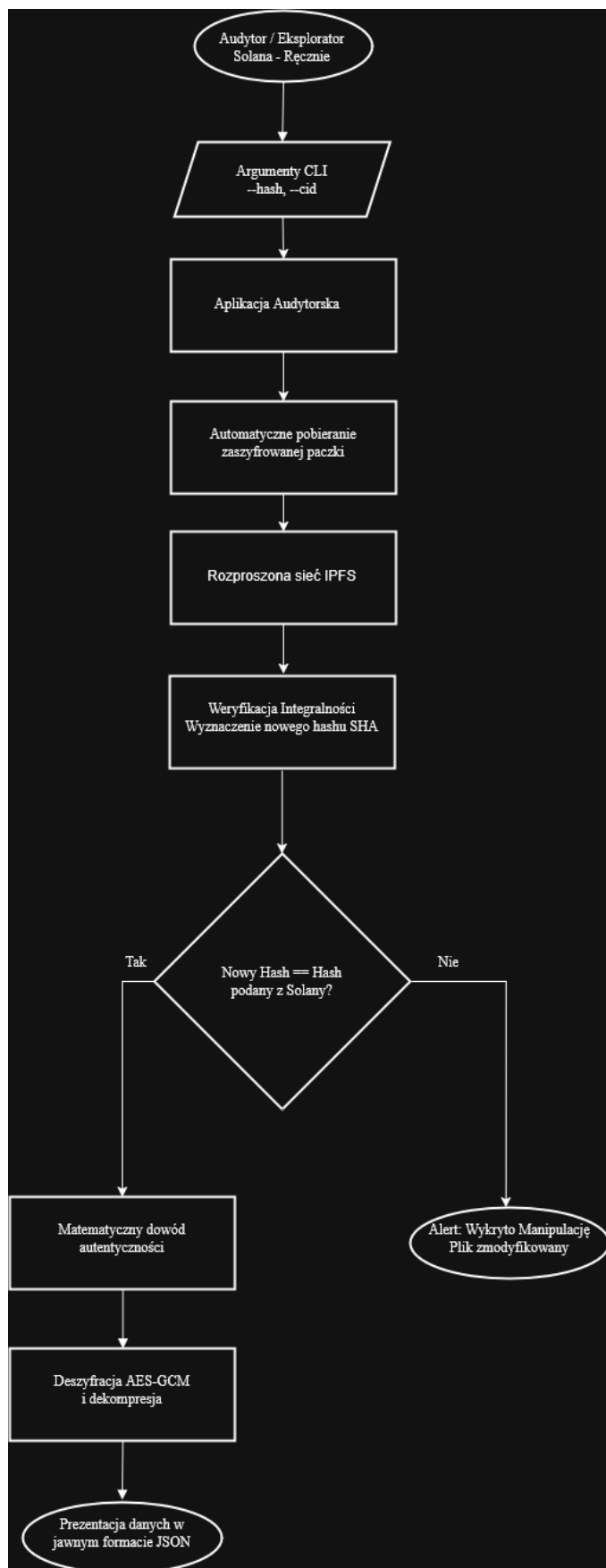
Bezpieczeństwo systemu opiera się na uznanych standardach kryptograficznych, chroniących dane przed nieautoryzowanym podglądem i modyfikacją. Zabezpieczenia zastosowano na całej ścieżce przepływu informacji produkcyjnej. Na etapie komunikacji z bramą urządzenia wykorzystują algorytm AES-256 w trybie CBC. Każdy wysyłany pomiar jest szyfrowany z użyciem unikalnego, losowego wektora inicjalizacyjnego, co zaciemnia ruch sieciowy i uniemożliwia atakującemu rozpoznanie powtarzających się wartości temperatury (np. stałego pomiaru 250°C).

Strumień archiwalny wymaga mechanizmów, które oprócz poufności zapewniają integralność. Zebrane logi są najpierw kompresowane algorytmem GZIP, a następnie poddawane szyfrowaniu uwierzytelnionemu AES-256 w trybie GCM, dzięki czemu każda manipulacja plikiem na serwerach IPFS kończy się błędem podczas deszyfracji. Ochronę uzupełnia skrót SHA-256 paczki, rejestrowany trwale w sieci Solana. Dane bieżące brama udostępnia jawnie przez Modbus TCP i WebUI. Jest to świadomy kompromis umożliwiający integrację z klasycznymi systemami SCADA w obrębie odizolowanej sieci zakładowej.

Całość domyka zewnętrzna procedura audytu, która uniezależnia przedsiębiorstwo od zaufania do własnych administratorów. Realizuje ją dedykowane oprogramowanie, którego działanie podzielono na trzy kroki:

1. Audytor dostarcza do programu identyfikator pliku IPFS (CID) oraz przypisany do niego skrót SHA-256, odczytane wcześniej z publicznego rejestru Solana.
2. Skrypt komunikuje się z systemem plików IPFS i pobiera z niego zaszyfrowaną paczkę z logami.
3. Aplikacja sprawdza integralność pliku, generując jego skrót i porównując go z wartością wejściową. Po potwierdzeniu zgodności zdejmuje warstwy kryptograficzne i prezentuje audytorowi dane w czytelnym formacie JSON.

W ten sposób powstaje środowisko niezaprzeczalności chroniące historię produkcyjną przed sabotażem. Proces przedstawiono na poniższym schemacie.



Rysunek 3 Schemat procedury audytorskiej i weryfikacji kryptograficznej

Źródło: opracowanie własne

ROZDZIAŁ IV. IMPLEMENTACJA SYSTEMU

4.1. Symulator urządzeń brzegowych

Pierwszym etapem prac wdrożeniowych było stworzenie oprogramowania obsługującego warstwę brzegową systemu. Ponieważ fizyczne podłączenie do rzeczywistej wtryskarki w warunkach laboratoryjnych było niemożliwe, zdecydowano się napisać symulator w języku Python. Program odwzorowuje zachowanie maszyny produkcyjnej, generując realistyczne odczyty temperatury i reagując na symulowane awarie. Całość zamknięto w środowisku kontenerowym Podman, co pozwala łatwo powielać instancje urządzeń bez ryzyka konfliktów w systemie operacyjnym.

Budowę aplikacji rozpoczęto od przygotowania importów i konfiguracji początkowej. Program korzysta ze standardowych bibliotek obsługujących sieć i czas oraz z zewnętrznych pakietów kryptograficznych zabezpieczających komunikację. W tej części kodu zdefiniowano stałe parametry pracy: adres sieciowy bramy, domyślny port komunikacyjny oraz współdzielony klucz szyfrujący. Klucz ma długość 256 bitów i został wygenerowany za pomocą bezpiecznego generatora liczb pseudolosowych. Konfigurację inicjującą działanie symulatora przedstawia poniższy listing.

Listing 4.1. Konfiguracja początkowa i importy bibliotek symulatora.

```
import socket
import json
import time
import random
import os
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
from dotenv import load_dotenv

load_dotenv()

cbc_key_str = os.getenv("AES_KEY_CBC")
if not cbc_key_str or len(cbc_key_str) != 32:
    raise ValueError("Klucz CBC musi mieć dokładnie 32 znaki!")

SECRET_KEY = cbc_key_str.encode('utf-8')

RUST_HOST = os.getenv('RUST_HOST', '172.17.0.1')
RUST_PORT = int(os.getenv('RUST_PORT', 65432))

CONTAINER_ID = socket.gethostname()
SENSOR_PHYS_ID = os.getenv('SENSOR_ID', 'sensor_X')
```

Źródło: opracowanie własne

Kolejnym etapem było opracowanie algorytmu odpowiadającego za matematyczny model wahań temperatury. Główna pętla programu symuluje ciągły napływ danych telemetrycznych. Aby odczyty nie były identyczne i sztuczne, wprowadzono parametr szumu pomiarowego, który nieznacznie i losowo je modyfikuje. W kodzie zaszyto uproszczoną logikę regulatora przemysłowego, utrzymującą temperaturę przy normalnej pracy w okolicy zadanej wartości 250°C (typowa temperatura wtrysku polipropylenu). Zaimplementowano też funkcję symulującą awarie: losowe, nagłe przegrzanie lub odcięcie zasilania grzałek. Logikę sterującą zachowaniem maszyny przedstawiono na poniższym listingu.

Listing 4.2. Pętla główna symulatora odpowiedzialna za model fizyczny i komunikację sieciową.

```
def run():
    print(f"Symulator wtryskarki | Kontener: {CONTAINER_ID} | Czujnik: {SENSOR_PHYS_ID}")

    current_temp = 250.0
    trend = 0.0

    while True:
        try:
            noise = random.uniform(-0.05, 0.05)

            if random.random() < 0.003:
                trend = random.uniform(3.0, 5.0)
            elif random.random() < 0.003:
                trend = random.uniform(-5.0, -3.0)
            elif random.random() < 0.01:
                trend = random.uniform(-0.3, 0.3)

            if current_temp > 251: trend -= 0.3
            if current_temp < 249: trend += 0.3

            trend *= 0.75

            current_temp += trend + noise

            payload = json.dumps({
                "container_id": str(CONTAINER_ID),
                "sensor_id": str(SENSOR_PHYS_ID),
                "wartosc": round(current_temp, 2)
            })

            encrypted_data = encrypt(payload)
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(3)
                s.connect((RUST_HOST, RUST_PORT))
                s.sendall(encrypted_data)

            time.sleep(random.uniform(3, 6))
```

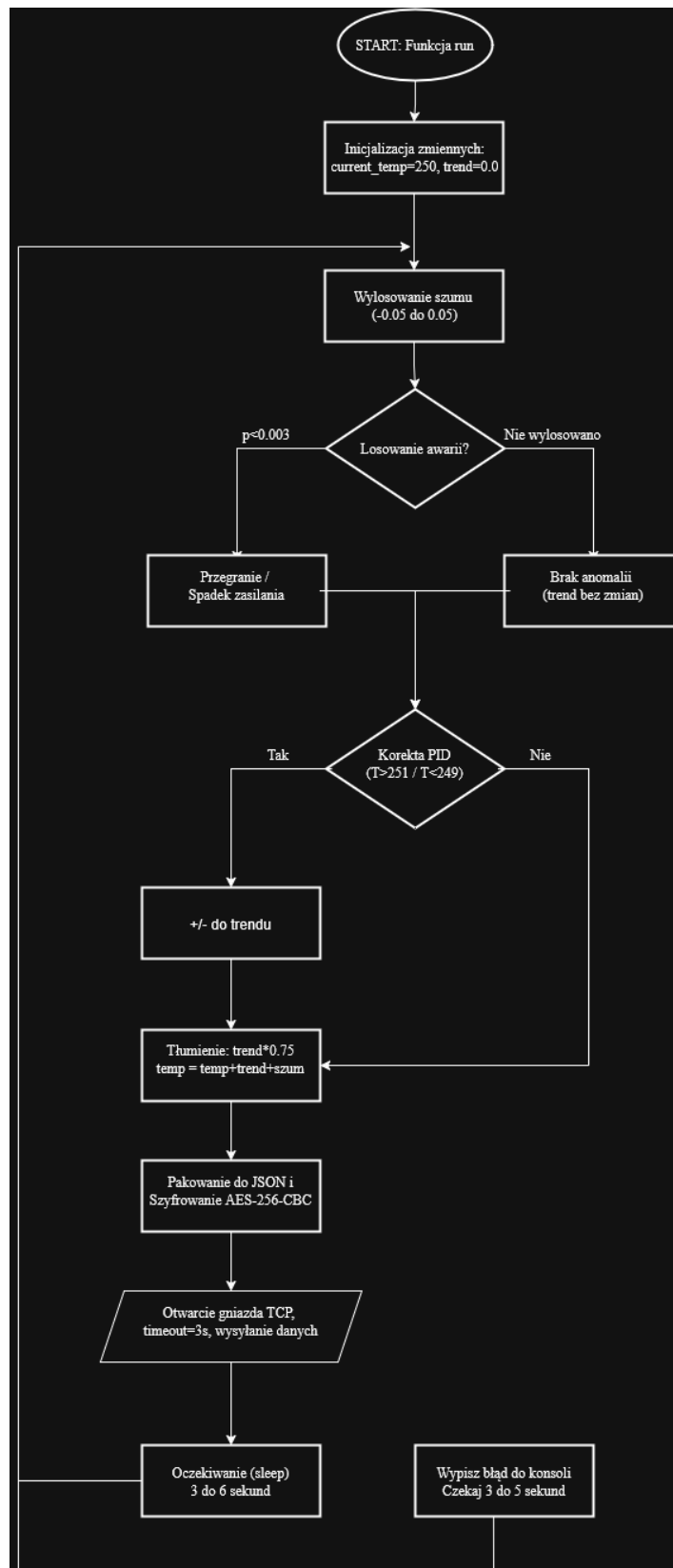


```
except Exception as e:
    print(f"[-] Błąd komunikacji brzegowej: {e}")
    time.sleep(random.uniform(3, 5))
```

Źródło: opracowanie własne

Po wyliczeniu temperatury dla danego cyklu dane są serializowane do struktury JSON zawierającej identyfikator maszyny, identyfikator czujnika oraz wartość pomiaru. Następnie funkcja encrypt zabezpiecza ładunek algorytmem AES w trybie CBC, a tak przygotowana paczka jest wysyłana do bramy integracyjnej protokołem TCP. Podczas początkowych prób integracyjnych napotkano problem blokowania się wątków symulatora, gdy brama była chwilowo niedostępna (np. w trakcie restartu usługi przy wgrywaniu nowej wersji kodu). Aby zapobiec trwałemu zawieszaniu kontenerów brzegowych, wprowadzono limit czasu połączenia wynoszący 3 sekundy oraz przechwytywanie wyjątków. Dzięki temu w razie awarii sieci maszyna nie blokuje procesu, lecz ponawia próbę wysyłki w kolejnym cyklu.

Aby zobrazować przebieg generowania danych, obsługi anomalii i komunikacji z warstwą nadrzędną, opracowano graficzną reprezentację algorytmu. Na poniższym schemacie blokowym przedstawiono etapy decyzyjne oraz operacje wejścia i wyjścia realizowane w pętli głównej symulatora urządzeń brzegowych.



Rysunek 4 Schemat działania pętli głównej symulatora urządzeń brzegowych

Źródło: opracowanie własne

4.2. Brama sieciowa

Pętla główna bramy sieciowej uruchamia jednocześnie niezależne wątki obsługujące serwer Modbus TCP, usługę archiwizatora IPFS oraz serwer panelu WebUI. Po ich inicjalizacji program przechodzi w stan ciągłego nasłuchu na gnieździe TCP przeznaczonym dla urządzeń brzegowych. Istotnym wyzwaniem na tym etapie budowy bramy, zaimplementowanej w języku Rust, było zarządzanie współbieżnością.

Ponieważ symulatory, serwer WebUI i archiwista działają jako niezależne wątki, próba jednoczesnego zapisu i odczytu logów początkowo kończyła się błędem blokady bazy SQLite (`database is locked`). Problem rozwiązano, wykonując instrukcje optymalizacyjne SQL (`PRAGMA`) i wymuszając tryb Write-Ahead Logging (WAL), który umożliwia równoległy zapis i odczyt. Dodatkowo, aby zapobiec wyścigom wątków w pamięci, zastosowano inteligentne wskaźniki Arc połączone z blokadami Mutex, co zapewnia atomowy i bezpieczny dostęp do bieżących stanów wtryskarek. Funkcję inicjującą oraz pętlę orkiestracji przedstawia poniższy listing.

Listing 4.3. Główna funkcja orkiestrująca bramę sieciową wraz z optymalizacją podsystemu bazy danych.

```
fn main() {
    println!("=====");
    println!("MATECHAIN BRAMA – Gateway + Modbus + Archiwista + WebUI");
    println!("=====");

    let conn = Connection::open(DB_PATH).expect("Błąd: Nie można otworzyć bazy danych SQLite.");
    conn.execute(
        "CREATE TABLE IF NOT EXISTS logi (
            id          INTEGER PRIMARY KEY,
            container_id TEXT,
            sensor_id   TEXT,
            wartosc      REAL,
            signature    TEXT,
            timestamp    DATETIME DEFAULT CURRENT_TIMESTAMP,
            batch_id     TEXT
        )",
        [],
    ).unwrap();

    conn.execute_batch(
        "PRAGMA journal_mode = WAL;
        PRAGMA synchronous = NORMAL;
```

```

        PRAGMA cache_size = -10000;
        PRAGMA temp_store = MEMORY;"
    ).unwrap();
    let shared_db = Arc::new(Mutex::new(conn));
    let rpc_url = String::from("https://api.devnet.solana.com");
    let client = Arc::new(RpcClient::new(rpc_url));
    let keypair_path = "/home/mati/.config/solana/id.json";
    let user_keypair = Arc::new(
        read_keypair_file(keypair_path).expect("Błąd: Nie znaleziono portfela So-
lana!")
    );
    let stany_maszyn: StanyMaszyn = Arc::new(Mutex::new(HashMap::new()));
    let etykiety: Etykiety = Arc::new(Mutex::new(HashMap::new()));

    let stany_dla_modbus = Arc::clone(&stany_maszyn);
    thread::spawn(move || { uruchom_serwer_modbus(stany_dla_modbus); });

    let db_dla_archiwisty = Arc::clone(&shared_db);
    let client_dla_archiwisty = Arc::clone(&client);
    let key_dla_archiwisty = Arc::clone(&user_keypair);
    thread::spawn(move || {
        uruchom_archiwiste(db_dla_archiwisty, client_dla_archiwisty, key_dla_archiwi-
sty);
    });
    let stany_dla_webui = Arc::clone(&stany_maszyn);
    let db_dla_webui = Arc::clone(&shared_db);
    let etykiety_dla_webui = Arc::clone(&etykiety);
    thread::spawn(move || {
        uruchom_serwer_webui(stany_dla_webui, db_dla_webui, etykiety_dla_webui);
    });
    let listener = TcpListener::bind("0.0.0.0:65432").expect("Błąd: Port 65432 za-
jęty.");

    for stream in listener.incoming() {
        match stream {
            Ok(s) => {
                let client_clone = Arc::clone(&client);
                let key_clone = Arc::clone(&user_keypair);
                let db_clone = Arc::clone(&shared_db);
                let stany_clone = Arc::clone(&stany_maszyn);
                let etykiety_clone = Arc::clone(&etykiety);
                thread::spawn(move || {
                    obsluz_kontener(s, client_clone, key_clone, db_clone,
stany_clone, etykiety_clone);
                });
            }
            Err(e) => println!("[!] Błąd połączenia TCP: {}", e),
        }
    }
}
}
}

```

Źródło: opracowanie własne

Współdzielenie połączenia z bazą danych (shared_db) oraz mapy stanów maszyn pomiędzy wątkami wymagało bezpiecznych struktur. Zastosowano w tym celu wskaźniki zliczające referencje (Arc) połączone z blokadami wzajemnego wykluczania (Mutex). Dzięki temu każdy wątek obsługujący konkretny kontener (obsluz_kontener) może bezpiecznie dla pamięci zapisywać odczyty do wspólnej bazy SQLite i aktualizować strukturę HashMap.

Centralnym elementem bramy jest funkcja przetwarzająca przychodzący strumień bajtów, przedstawiona na listingu 4.4. Odczytuje ona surowy bufor, wyodrębnia wektor inicjalizacyjny zajmujący pierwsze 16 bajtów pakietu i deszyfruje pozostałą część komunikatu. Proces realizuje algorytm AES-256-CBC z dopełnieniem PKCS7, zapewniający poprawne zdekodowanie całego ładunku.

Listing 4.4. Logika dekapulacji pakietu sieciowego, deszyfracji AES-CBC oraz klasyfikacji zdarzeń.

```
fn obsluz_kontener(
    mut stream: TcpStream,
    client: Arc<RpcClient>,
    user_keypair: Arc<Keypair>,
    db: Arc<Mutex<Connection>>,
    stany: StanyMaszyn,
    etykiety: Etykiety,
) {
    let mut buffer = [0; 1024];

    if let Ok(size) = stream.read(&mut buffer) {
        if size < 17 { return; }

        let iv_array: [u8; 16] = match buffer[..16].try_into() {
            Ok(arr) => arr,
            Err(_) => return,
        };
        let ciphertext = &buffer[16..size];

        let decryptor = Decryptor::<Aes256>::new(AES_KEY_CBC.into(), (&iv_array).into());
        let mut out_buffer = ciphertext.to_vec();

        match decryptor.decrypt_padded_mut::<Pkcs7>(&mut out_buffer) {
            Ok(decrypted) => {
                let msg = String::from_utf8_lossy(decrypted);

                match serde_json::from_str::<SensorData>(&msg) {
                    Ok(sensor) => {
                        let numer_czujnika = {
                            let mut etyk = etykiety.lock().unwrap();
                            if let Some(&n) = etyk.get(&sensor.container_id) { n }
                            else {
                                let nowy = etyk.len() as u32 + 1;
                                etyk.insert(sensor.container_id.clone(), nowy);
                                nowy
                            }
                        };

                        let is_alarm = sensor.value > 253.0 || sensor.value < 247.0;
                        let mut sig_val: Option<String> = None;

                        if is_alarm {
                            if let Ok(sig) = wyslij_alarm_na_solane(sensor.clone(), numer_czujnika, &client, &user_keypair) {
                                sig_val = Some(sig);
                            }
                        }
                    }
                }
            }
        }
    }
}
```

```

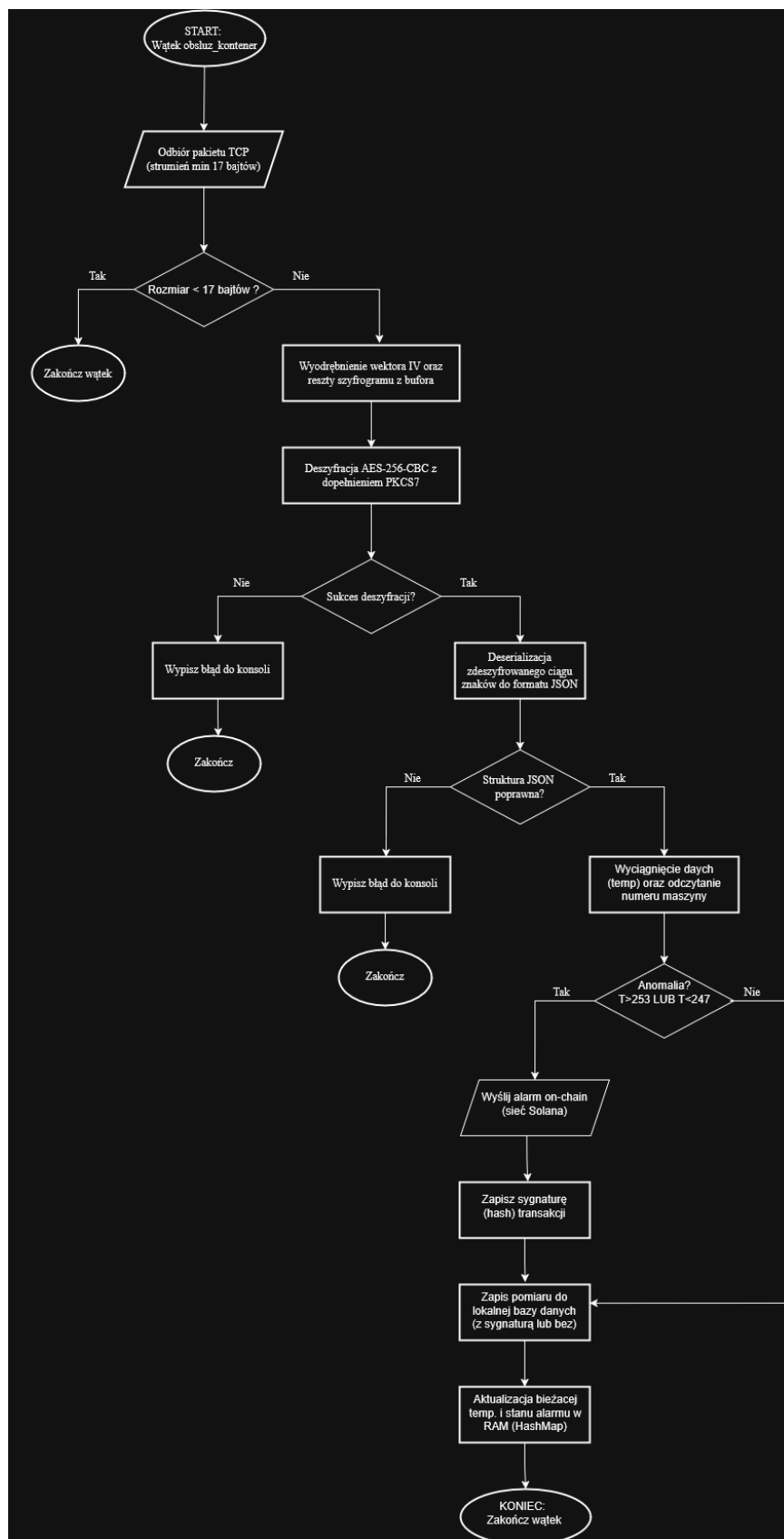
        zapisz_do_sql(&sensor, sig_val, &db);

        if let Ok(mut mapa) = stany.lock() {
            mapa.insert(sensor.container_id.clone(), StanWtryskarki {
                temp: sensor.value,
                alarm: is_alarm,
            });
        }
    }
    Err(_) => println!("[!] Błąd parsowania JSON"),
}
}
Err(e) => println!("[!] Błąd deszyfracji AES: {:?}" , e),
}
}
}

```

Źródło: opracowanie własne

Po zdeserializowaniu struktury JSON brama realizuje kluczowe założenie projektowe czyli dwutorową klasyfikację zdarzeń. Instrukcja warunkowa sprawdza, czy odczytana temperatura przekracza dopuszczalne granice technologiczne (spadek poniżej 247°C lub wzrost powyżej 253°C). W razie wykrycia anomalii program wywołuje funkcję `wyslij_alarm_na_solane`, realizującą komunikację on-chain w czasie rzeczywistym (Tor 1). Niezależnie od wyniku tej oceny każdy odczyt jest trwale zapisywany w lokalnej bazie za pomocą funkcji `zapisz_do_sql`, co przygotowuje dane dla Toru 2 czyli cyklicznej archiwizacji off-chain. Przepływ decyzyjny bramy przedstawiono na poniższym schemacie blokowym.



Rysunek 5 Schemat blokowy logiki klasyfikacji zdarzeń i dekapulacji danych w bramie sieciowej

Źródło: opracowanie własne

4.3. Inteligentny kontrakt w sieci Solana

Inteligentny kontrakt zaimplementowano przy użyciu frameworka Anchor, stanowiącego standard tworzenia programów dla ekosystemu Solana. Framework ten abstrahuje wiele niskopoziomowych mechanizmów, pozwalając programiście skupić się na logice działania i ogranicza ryzyko błędów pamięciowych. Kompletny kod kontraktu przedstawiono na poniższym listingu.

Listing 4.5. Główna struktura inteligentnego kontraktu w środowisku Anchor

```
use anchor_lang::prelude::*;

declare_id!("3zcbbgzgNqWLyTUIJKzuzzT9t1t3VpdcuSQw9vA5qj3Zm");

#[program]
pub mod kontrakt {
    use super::*;

    pub fn log_event(ctx: Context<LogEvent>, message: String) -> Result<()> {
        let event_account = &mut ctx.accounts.event_account;
        event_account.message = message.clone();
        event_account.timestamp = Clock::get()?.unix_timestamp;

        msg!("Zapisano alarm: {}", message);
        Ok(())
    }

    pub fn log_batch(
        ctx: Context<LogBatch>,
        cid: String,
        content_hash: String,
        records_count: u32,
    ) -> Result<()> {
        let batch_account = &mut ctx.accounts.batch_account;
        batch_account.cid = cid.clone();
        batch_account.content_hash = content_hash;
        batch_account.records_count = records_count;
        batch_account.timestamp = Clock::get()?.unix_timestamp;

        msg!("Zapisano paczkę: CID={} rekordów={}", cid, records_count);
        Ok(())
    }
}

#[derive(Accounts)]
pub struct LogEvent<'info> {
    #[account(init, payer = user, space = 8 + 300)]
    pub event_account: Account<'info, EventData>,
    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

#[derive(Accounts)]
pub struct LogBatch<'info> {
```



```

    #[account(init, payer = user, space = 8 + 100 + 80 + 4 + 8 + 100)]
    pub batch_account: Account<'info, BatchData>,
    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

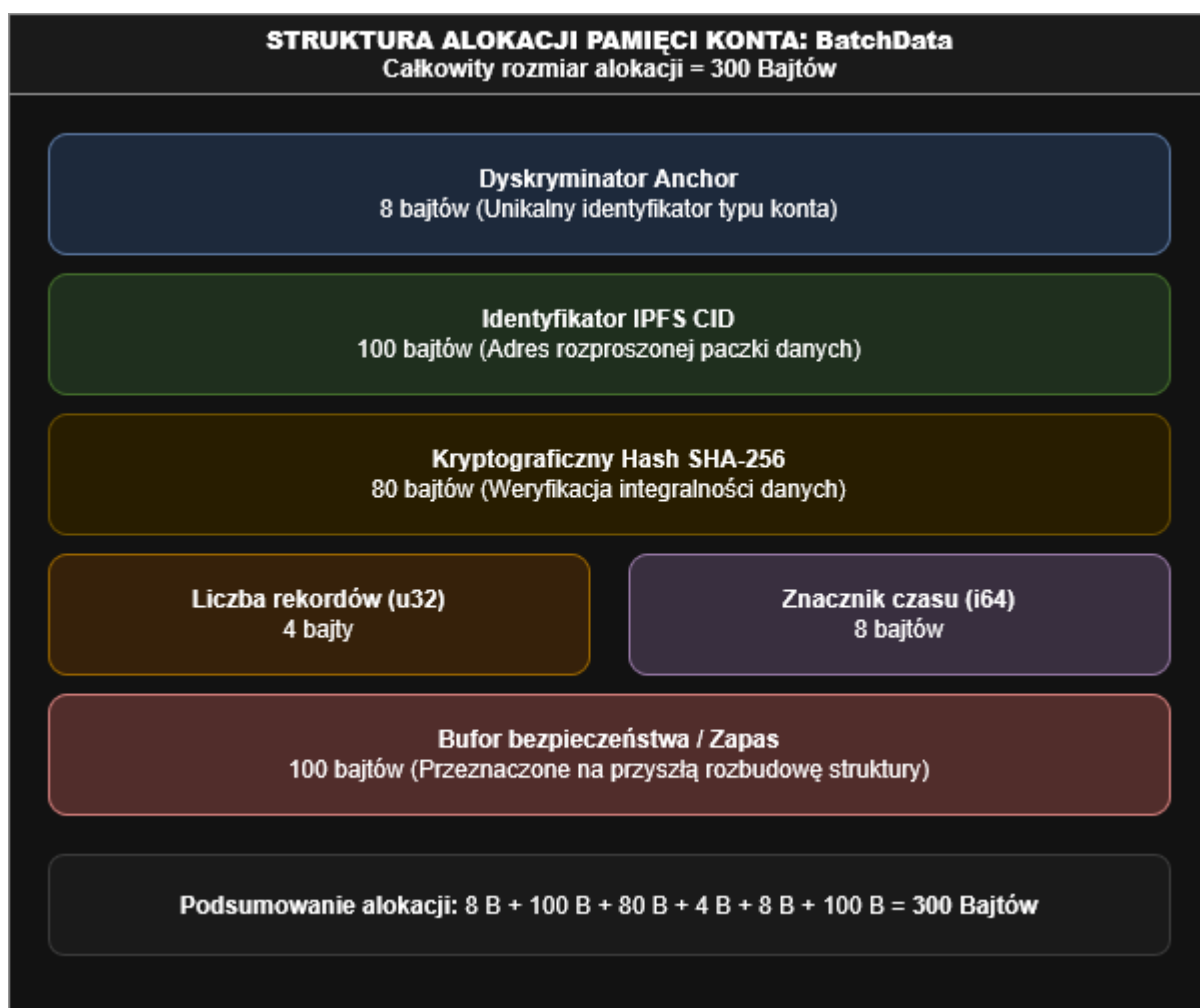
#[account]
pub struct EventData {
    pub message: String,
    pub timestamp: i64,
}

#[account]
pub struct BatchData {
    pub cid: String,
    pub content_hash: String,
    pub records_count: u32,
    pub timestamp: i64,
}

```

Źródło: opracowanie własne

Kontrakt opiera się na dwóch strukturach danych (EventData i BatchData), odpowiadających dwutorowej strategii rejestracji zdarzeń zdefiniowanej wcześniej. Każde wywołanie bramy (funkcją `log_event` lub `log_batch`) tworzy nowe konto w pamięci łańcucha bloków za pomocą instrukcji `init`. Najważniejszym wyzwaniem jest tu ręczne zdefiniowanie rozmiaru przydzielanej pamięci, co pozwala zoptymalizować opłaty w walucie SOL. W strukturze `LogBatch` alokacja składa się z: 8 bajtów na dyskryminator narzędzia `Anchor`, 100 bajtów na identyfikator `CID`, 80 bajtów na skrót `SHA-256`, 4 bajtów na liczbę rekordów (typ `u32`), 8 bajtów na znacznik czasu (`i64`) oraz dodatkowego bufora. Daje to rozwiązanie skalowalne ekonomicznie i zarazem chroniące dane składowane off-chain. Model alokacji pamięci dla konta `BatchData` przedstawiono na poniższym schemacie.



Rysunek 6 Schemat rozkładu pamięci i alokacji struktury danych konta BatchData w programie Solana

Źródło: opracowanie własne

4.4. Aplikacja audytorska i weryfikacja danych

Skrypt audytorski wykorzystuje standardowe struktury języka Python oraz bibliotekę kryptograficzną pycryptodome. Poniższy listing przedstawia funkcje odpowiedzialne za pobieranie danych binarnych z sieci rozproszonej oraz weryfikację ich integralności algorytmem SHA-256.

Listing 4.6. Implementacja funkcji pobierających dane z sieci IPFS oraz weryfikacji skrótu SHA-256.

```
def pobierz_z_ipfs(cid, uzyj_publicznej_bramy=False):
    if not uzyj_publicznej_bramy:
        url = f"{IPFS_LOCAL_API}/api/v0/cat?arg={cid}"
    try:
```

```

        req = urllib.request.Request(url, method="POST")
        with urllib.request.urlopen(req, timeout=10) as response:
            return response.read()
    except Exception as e:
        blad(f"Lokalny węzeł nie odpowiada: {e}")
        print(f" {CLR_YELLOW}→ Próbuje publicznej bramy...{CLR_RESET}")
        uzyj_publicznej_bramy = True

if uzyj_publicznej_bramy:
    url = f"{IPFS_GATEWAY_PUBLIC}/{cid}"
    try:
        with urllib.request.urlopen(url, timeout=30) as response:
            return response.read()
    except Exception as e:
        raise RuntimeError(f"Nie udało się pobrać paczki: {e}")

def weryfikuj_hash(dane, oczekiwany_hash):
    rzeczywisty_hash = hashlib.sha256(dane).hexdigest()
    info("Oczekiwany SHA-256:", oczekiwany_hash, CLR_CYAN)
    info("Obliczony SHA-256:", rzeczywisty_hash, CLR_CYAN)

    if rzeczywisty_hash.lower() == oczekiwany_hash.lower():
        sukces("Skróty się zgadzają – paczka jest autentyczna")
        return True
    else:
        blad("Skróty się NIE zgadzają – paczka mogła zostać zmodyfikowana!")
        return False

```

Źródło: opracowanie własne

W trakcie pierwszych testów procedury audytorskiej zaobserwowano, że lokalny demon IPFS sporadycznie odrzuca połączenia na porcie API z powodu przeciążenia routingiem w tablicy DHT. Aby zwiększyć niezawodność audytu, funkcję `pobierz_z_ipfs` rozbudowano o obsługę awarii sieciowych. W pierwszej kolejności aplikacja próbuje połączyć się z lokalnym węzłem IPFS zapytaniem HTTP POST; jeśli w ciągu 10 sekund nie nawiąże połączenia, automatycznie przełącza się na tor rezerwowy i pobiera paczkę z publicznej bramy (np. `ipfs.io`) metodą GET. Zapewnia to ciągłość kontroli nawet przy czasowej awarii lokalnej infrastruktury. Funkcja `weryfikuj_hash` realizuje natomiast założenie niezaprzeczalności: wylicza skrót SHA-256 z pobranego strumienia bajtów i porównuje go z wartością odczytaną z łańcucha bloków.

Po potwierdzeniu integralności pliku aplikacja audytorska przystępuje do zdjęcia warstw kryptograficznych chroniących tajemnicę przedsiębiorstwa. Ponieważ dane archiwalne zapisano w trybie uwierzytelnionego szyfrowania AES-256-GCM, skrypt musi poprawnie rozdzielić metadane pakietu. Proces ten, połączony z dekompresją strumienia danych, przedstawiono na listingu 4.7.

Listing 4.7. Logika podziału szyfrogramu, deszyfracji AES-GCM oraz dekompresji GZIP.

```
def odszyfruj_paczke(zaszyfrowane_dane):
    nonce = zaszyfrowane_dane[:12]
    ciphertext = zaszyfrowane_dane[12:-16]
    tag = zaszyfrowane_dane[-16:]

    try:
        cipher = AES.new(AES_KEY_GCM, AES.MODE_GCM, nonce=nonce)
        plaintext = cipher.decrypt_and_verify(ciphertext, tag)
        sukces("Odszyfrowano poprawnie – tag GCM zweryfikowany")
        return plaintext
    except ValueError as e:
        blad(f"Błąd autentyczności GCM: {e}")
        raise

def rozpakuj_gzip(skompresowane):
    rozpakowane = gzip.decompress(skompresowane)
    return rozpakowane
```

Źródło: opracowanie własne

Podczas deszyfracji istotne jest dokładne wyodrębnienie składowych z tablicy bajtów. Pierwsze 12 bajtów to wektor nonce, ostatnie 16 bajtów to tag autentyczności wygenerowany przez bramę, a środkowa część stanowi właściwy szyfrogram z danymi produkcyjnymi. Metoda `decrypt_and_verify` sprawia, że jeśli na serwerze IPFS zmodyfikowano lub podmieniono jakikolwiek bit pliku, funkcja zgłasza wyjątek `ValueError`. Dla audytora jest to matematyczny dowód próby manipulacji historią produkcyjną i powód do automatycznego przerwania kontroli. Po udanej deszyfracji następuje dekompresja algorytmem GZIP, która przywraca surowy plik w formacie JSON Lines, gotowy do analizy i parsowania przez kontrolera.

4.5. Panel operatorski WebUI

Ostatnim etapem prac w warstwie integracyjnej było stworzenie graficznego interfejsu użytkownika. Zgodnie z założeniami projektowymi system ma umożliwiać personelowi technicznemu i operatorom bieżący podgląd parametrów pracy maszyn w czasie rzeczywistym, bez sięgania do bazy danych czy rozproszonego rejestru. Zadanie to realizuje panel WebUI, serwowany przez bramę sieciową na porcie 8090.

Serwer interfejsu zaimplementowano jako niezależny, nieblokujący wątek w aplikacji napisanej w języku Rust. Wykorzystano lekki framework `tiny_http`, co ograniczyło narzut pamięciowy i pozwoliło uniknąć ciężkich, zewnętrznych serwerów aplikacyjnych.

Moduł obsługuje dwa rodzaje żądań: serwuje statyczne pliki frontowe (HTML oraz skrypty JavaScript z biblioteką Chart.js do renderowania wykresów) oraz udostępnia punkt końcowy API w formacie JSON, z którego przeglądarka cyklicznie pobiera aktualne stany wtryskarek. Funkcję obsługującą serwer przedstawia poniższy listing.

Listing 4.8. Implementacja serwera HTTP w języku Rust odpowiedzialnego za obsługę panelu WebUI oraz API JSON.

```
pub fn uruchom_serwer_webui(stany: StanyMaszyn, db: Arc<Mutex<Connection>>, etykiety:
Etykiety) {
    let server = tiny_http::Server::http("0.0.0.0:8090").expect("Błąd: Serwer WebUI nie
mógł wystartować.");
    println!("[+ WEBUI] Panel operatorski uruchomiony pod adresem http://lo-
calhost:8090");

    for request in server.incoming_requests() {
        let url = request.url().to_string();
        if url == "/api/status" {
            let mapa_stanow = stany.lock().unwrap();
            let mut json_wynik = String::from("{}");

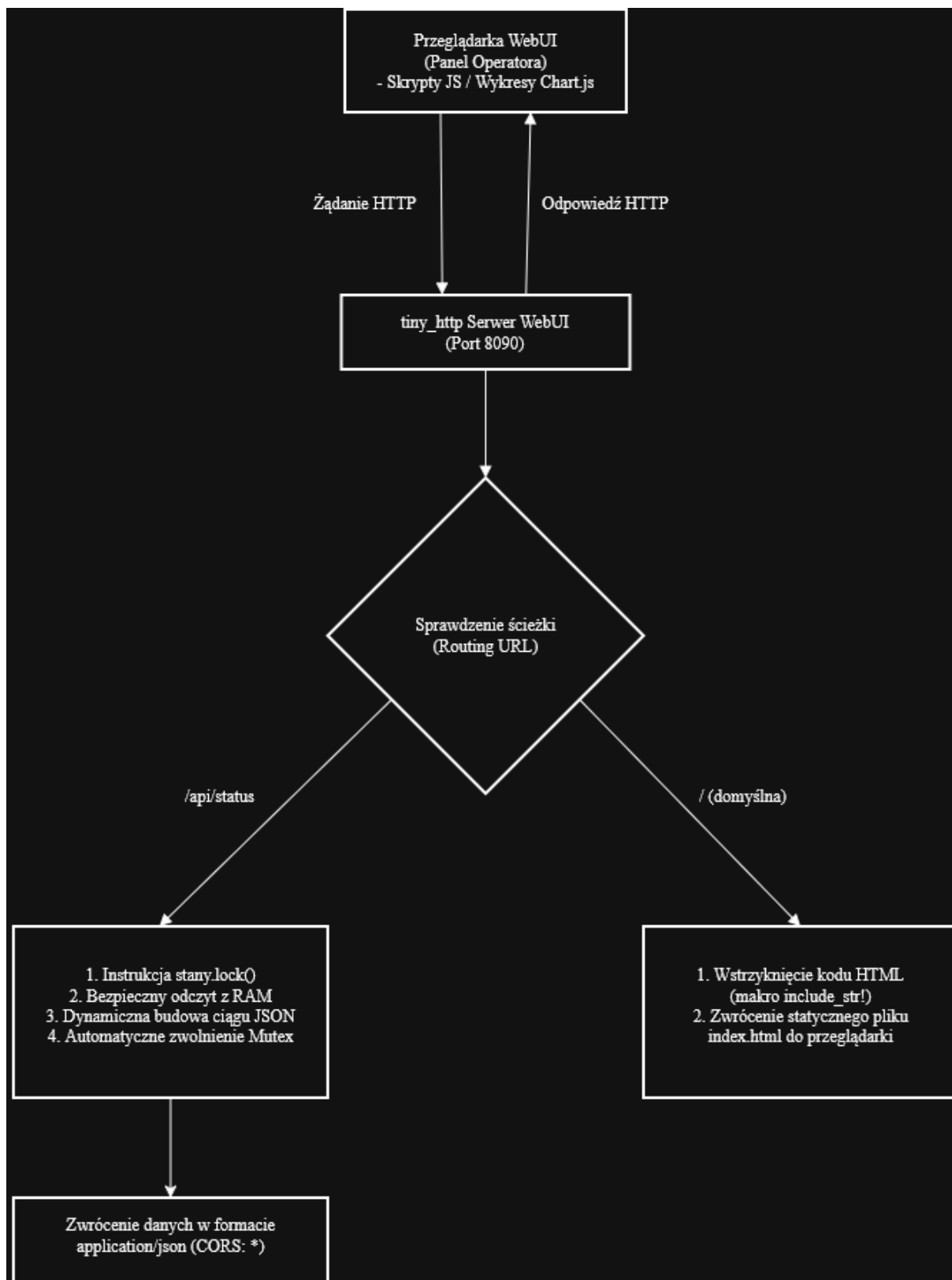
            for (idx, (container_id, stan)) in mapa_stanow.iter().enumerate() {
                if idx > 0 { json_wynik.push_str(","); }
                json_wynik.push_str(&format!(
                    "\\{}\\": {{\\\"temp\\\": {}, \\\"alarm\\\": {}}}",
                    container_id, stan.temp, stan.alarm
                ));
            }
            json_wynik.push_str("{}");

            let response = tiny_http::Response::from_string(json_wynik)
                .with_header(tiny_http::Header::from_bytes(&"Content-Type"[..],
&"application/json"[..]).unwrap())
                .with_header(tiny_http::Header::from_bytes(&"Access-Control-Allow-
Origin"[..], &"*"[..]).unwrap());
            let _ = request.respond(response);
        } else {
            let html_content = include_str!("index.html");
            let response = tiny_http::Response::from_string(html_content)
                .with_header(tiny_http::Header::from_bytes(&"Content-Type"[..],
&"text/html; charset=utf-8"[..]).unwrap());
            let _ = request.respond(response);
        }
    }
}
```

Źródło: opracowanie własne

Istotnym elementem implementacji serwera jest bezpieczeństwo pamięciowe podczas współdzielenia danych. Funkcja `uruchom_serwer_webui` otrzymuje wskaźniki `Arc` do struktur z bieżącymi stanami wtryskarek. Po nadejściu zapytania pod adres `/api/status` wątek serwera na ułamek milisekundy blokuje strukturę `HashMap` instrukcją `stany.lock().unwrap()`, bezpiecznie odczytuje temperatury zapisane przez wątki

obsługujące kontenery, po czym natychmiast zwalnia blokadę. Dane są następnie serializowane do formatu JSON i wysyłane do klienta wraz z nagłówkami zezwalającymi na współdzielenie zasobów między domenami. W przypadku żądania strony głównej serwer korzysta z makra `include_str!`, które wstrzykuje zawartość pliku `index.html` do skompilowanego pliku binarnego bramy. Eliminuje to potrzebę dystrybucji dodatkowych plików i sprawia, że panel działa niezależnie od ścieżek w systemie operacyjnym komputera brzegowego.



Rysunek 7 Schemat przepływu danych i architektury routingu serwera panelu operatorskiego

Źródło: opracowanie własne

ROZDZIAŁ V. TESTY I ANALIZA WYNIKÓW

5.1. Środowisko testowe i konfiguracja infrastruktury

Weryfikacja funkcjonalna, wydajnościowa i ekonomiczna systemu MateChain wymagała zorganizowania dedykowanego środowiska testowego, które w warunkach laboratoryjnych odzwierciedla strukturę rozproszonej sieci fabrycznej. Stanowisko badawcze uruchomiono na komputerze przenośnym HP EliteBook 725 G3, wyposażonym w czterordzeniowy procesor AMD A12-8800B, 8 GB pamięci RAM DDR3 oraz dysk SSD. Taka konfiguracja pozwala odwzorować wydajność niskobudżetowych, energooszczędnych komputerów przemysłowych klasy IPC, powszechnie stosowanych na halach produkcyjnych jako bramy brzegowe w architekturze Przemysłu 4.0. Systemem operacyjnym stacji była dystrybucja Zorin OS oparta na jądrze Linux, co zapewniło stabilne warunki dla wirtualizacji sieciowej i izolacji zasobów.

Rdzeniem topologii testowej była infrastruktura zorganizowana za pomocą narzędzia orkiestracji wielokontenerowej na podstawie pliku konfiguracyjnego `compose.yml`. W wydzielonej sieci lokalnej uruchomiono cztery niezależne, odseparowane na poziomie systemu operacyjnego komponenty. Pierwszą grupę stanowiły trzy kontenery reprezentujące warstwę brzegową: `wtryskarka_1`, `wtryskarka_2` oraz `wtryskarka_3`. Każdy z nich wykonywał osobną instancję pythonowego symulatora procesu i miał przypisany unikalny identyfikator czujnika, oznaczony odpowiednio jako `termopara_PT100_1`, `termopara_PT100_2` i `termopara_PT100_3`. Czwartym elementem był oficjalny obraz rozproszonego systemu plików IPFS Kubo, pełniący rolę zdecentralizowanego magazynu off-chain dla drugiego toru rejestracji logów.

Istotnym zadaniem na etapie konfiguracji było zapewnienie poprawnej komunikacji sieciowej między izolowanymi kontenerami a bramą integracyjną, uruchomioną bezpośrednio na hoście jako proces skompilowany w języku Rust. Ze względu na specyfikę sieci kontenerowej w systemie Linux maszyny brzegowe komunikowały się z bramą przez mechanizm mapowania adresu IP hosta (`host.docker.internal`), przesyłając zaszyfrowane ładunki AES-256-CBC na port 65432 systemu bazowego.

Brama realizowała wielowątkowe połączenia z lokalnym węzłem IPFS Kubo przez wystawione porty API: port 5001 obsługiwał zapytania administracyjne HTTP POST, a porty 8080 i 8081 pełniły funkcję bram odczytu plików. Na potrzeby integracji z automatyką przemysłową i systemami wizualizacji brama nasłuchiwała na porcie 5502 (Modbus TCP) oraz na porcie 8090 (panel WebUI). Zewnętrzny komponent audytorski (audytor.py) wykonywał wywołania klienckie HTTP w celu weryfikacji danych off-chain, pobierając metadane transakcyjne z publicznego klastra testowego Solana devnet.

Mapowanie interfejsów oraz relacje sieciowe między modułami w trakcie testów zestawiono w poniższej tabeli.

Tabela 1 Konfiguracja interfejsów sieciowych i mapowania portów środowiska testowego

Nazwa komponentu	Środowisko uruchomieniowe	Protokół warstwy aplikacji	Port lub adres	Rola w architekturze systemu
Brama Mate-Chain	Host (ZorinOs/Rust)	Autorski	0.0.0.0:65432	Odbiór logów z maszyn brzegowych
Brama Modbus	Host (ZorinOs/Rust)	Modbus TCP	0.0.0.0:5502	Integracja z systemami SCADA
Panel WebUI	Host (ZorinOs/Rust)	HTTP/JSON	0.0.0.0:8090	Wizualizacja bieżących stanów i wykresów
Węzeł IPFS	Kontener Podman	HTTP API/Libp2p	127.0.0.1:5001	Decentralizacja i przechowywanie paczek off-chain
Symulator wtryskarek	Kontener Podman(Python)	Klient TCP	Wyjście na host-gateway	Generowanie i szyfrowanie danych pomiarowych
Solana Devnet	Chmura Publiczna (Devnet)	JSON-RPC	api.devnet.solana.com	Warstwa globalnego konsensusu i niezaprzeczalności

Źródło: opracowanie własne

5.2. Weryfikacja funkcjonalna systemu

Celem testów funkcjonalnych było potwierdzenie poprawnego działania dwutorowej strategii rejestracji zdarzeń. W środowisku testowym uruchomiono wszystkie kontenery warstwy brzegowej oraz proces głównej bramy sieciowej. Symulatory wtryskarek rozpoczęły cykliczne generowanie zaszyfrowanych pakietów telemetrycznych, które trafiły do bramy przez lokalny interfejs sieciowy.

5.2.1. Rejestracja anomalii w czasie rzeczywistym

Pierwszy scenariusz testowy weryfikował czas reakcji systemu na nagłą anomalię technologiczną. W jednym z kontenerów (wtryskarka_2) algorytm wylosował zdarzenie symulujące awarię termostatu, co doprowadziło do spadku temperatury poniżej dopuszczalnego progu 247°C.

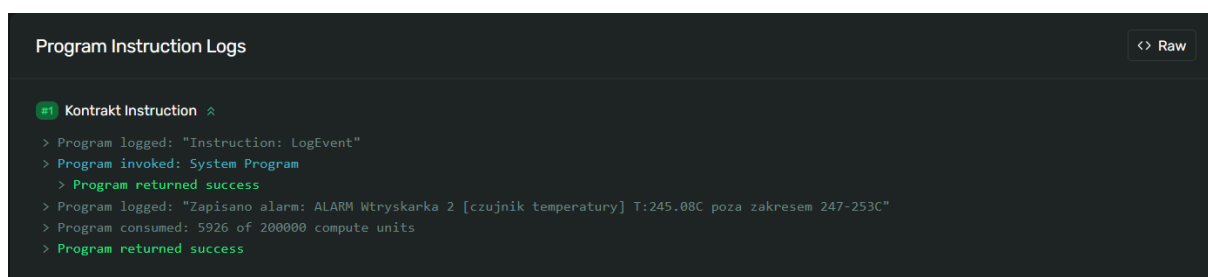
Brama odebrała zaszyfrowany ładunek, wyodrębniła wektor inicjalizacyjny i przeprowadziła deszyfrację algorytmem AES-CBC. Klasyfikator zidentyfikował przekroczenie wartości brzegowych, a brama wygenerowała krytyczny komunikat w terminalu operatorskim i natychmiast sformułowała transakcję, kierując ją do klastra Solana devnet. Przebieg tego procesu z perspektywy konsoli centralnej przedstawiono na poniższym zrzucie ekranu.

```
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 243.74°C
>>> [ SOLANA OK ] Sygnatura: 3vN71DaNEjUHprYSfJ8UXDenAnwRW2wDvypG2LhPw61ksULGV9ZngMfPn4ary9aE8Md32Hi5dkSy3dVMXj3PhmW
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.56°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.75°C
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 244.00°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.54°C
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 244.49°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.76°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.49°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.75°C
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 245.08°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.54°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.71°C
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 245.69°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.57°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.72°C
[ 🚨 ALARM ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 246.36°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.57°C
[ ✅ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.75°C
[ ✅ OK ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 247.15°C
[ ✅ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.54°C
>>> [ SOLANA OK ] Sygnatura: Th3TiQtG93BBNE652QbdUB5hfW5Atjcn6WcguHNraCmGsY7fcQVR4xs9iqrqDyha6uG2B95q1JTZFuqCPGB2oEur
>>> [ SOLANA OK ] Sygnatura: ppsmJjmeMnd3qfy211STzbYRjpszoZEWmdAuLaDEPhUTa6EpPbKY6KEXjVgoUPUoBUbaT2ot2M1BzFP4PiVmBV
>>> [ SOLANA OK ] Sygnatura: 59Jqn19v6kyfMHyHu9tLHrsFdgalJGhjRy6tMdKd1DjDyDMehwVPD4ZjyqKdihngMX1C8uDLJF8qRVmM6HMj3K3c
>>> [ SOLANA OK ] Sygnatura: 4VPBzybFm9KNZpzmynjLUdMz1T1KW2mJNorT88gtv8XL32NvxHCctiwvcwhUBKoeHabyGHfQwvFtZfgWBA8yRqic
```

Rysunek 8 Wykrycie anomalii temperaturowej oraz wygenerowanie sygnatury transakcji w terminalu bramy

Źródło: opracowanie własne

Dla potwierdzenia niezaprzeczalności zdarzenia uzyskaną sygnaturę transakcji sprawdzono w publicznym eksploratorze bloków sieci Solana. W szczegółach transakcji znalazł się wygenerowany przez framework Anchor dyskryminator funkcji `log_event` oraz czytelny ciąg znaków precyzujący źródło i powód alarmu. Czas od wykrycia awarii do zatwierdzenia bloku w rejestrze rozproszonym wyniósł ułamek sekundy, co spełnia wymagania stawiane systemom czasu rzeczywistego w Przemysle 4.0.



Rysunek 9 Zarejestrowany alarm krytyczny w publicznym eksploratorze sieci Solana (klaster devnet)

Źródło: opracowanie własne

5.2.2. Cykliczna archiwizacja danych i integracja z IPFS

Drugi scenariusz dotyczył procesu ciągłej, ekonomicznej archiwizacji danych produkcyjnych czyli mechanizmu kluczowego dla optymalizacji kosztów utrzymania infrastruktury łańcucha bloków.

Podczas nieprzerwanej pracy symulatorów brama gromadziła odczyty w lokalnej bazie SQLite działającej w trybie WAL. Po upływie zaprogramowanego interwału 120 sekund wybudzany był niezależny wątek archiwisty. Pobierał on nieoznaczone rekordy z bazy i serializował je do formatu JSON Lines. Następnie zbiór poddawano kompresji algorytmem GZIP oraz szyfrowaniu uwierzytelnionemu AES-GCM, co chroni tajemnicę przedsiębiorstwa przed dostępem osób trzecich w publicznej sieci peer-to-peer²⁶.

²⁶ SQLite Documentation — Write-Ahead Logging, oficjalna dokumentacja techniczna, dostęp online: <https://www.sqlite.org/wal.html> [dostęp: 4.06.2026].

Przygotowana paczka binarna trafiała do lokalnego węzła IPFS Kubo przez interfejs API (port 5001). W odpowiedzi system otrzymywał unikalny identyfikator treści (CID), który wraz z wyliczonym wcześniej skrótem SHA-256 całego archiwum był wysyłany jako pojedyncza transakcja do inteligentnego kontraktu Solana metodą `log_batch`.

```
[✓ OK ] Czujnik 1 | Kontener: 58ff38d395fe | Temp: 250.22°C
[✓ OK ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 249.28°C
[ARCHIWISTA] Paczka wysłana: CID=QmNo314979DxuvGocMivQoFYLv1kE19YMfytpkyvLf583Y rekordów=169
[✓ OK ] Czujnik 3 | Kontener: 5fb29ea67bb5 | Temp: 249.92°C
[✓ OK ] Czujnik 2 | Kontener: 4ffb6957edf1 | Temp: 249.39°C
```

Rysunek 10 Proces agregacji i wysyłki paczki historycznej do systemu IPFS

Źródło: opracowanie własne

Ostatnim etapem archiwizacji była aktualizacja rekordów w lokalnej bazie SQLite. Kolumnę `batch_id` zarchiwizowanych pomiarów uzupełniano nową sygnaturą, co zapobiega dublowaniu danych w kolejnych iteracjach wątku. Test potwierdził płynne współdziałanie trzech niezależnych technologii: wielowątkowej aplikacji w języku Rust, klastra IPFS oraz zewnętrznej sieci blockchain.

5.2.3. Wymiana danych za pomocą protokołu Modbus TCP

Ostatnim elementem weryfikacji funkcjonalnej było przetestowanie interfejsu komunikacyjnego od strony nadrzędnych systemów sterowania produkcją. Zgodnie z założeniami projektowymi brama MateChain ma zachować pełną transparentność i wsteczną kompatybilność z klasycznymi systemami klasy SCADA, dlatego wykorzystano uznany w automatyce przemysłowej protokół Modbus TCP.

Podczas pracy symulatorów zainicjowano połączenie z zewnętrznego klienta testowego do bramy na porcie 5502. Klient wysłał najpierw żądanie odczytu rejestrów wejściowych, aby pobrać bieżące wartości temperatury ze wszystkich trzech wtryskarek, a następnie żądanie odczytu cewek w celu weryfikacji flag alarmowych. Bieżące parametry produkcji odczytane przez klienta Modbus przedstawiono na poniższym zrzucie ekranu.

```

mati@mat1-HP-EliteBook-725-G3:~/Pulpit/Praca_Dyplomowa$ python3 test_modbus.py
--- TEST PROTOKOŁU MODBUS TCP ---
✓ Połączenie udane. Pobrane rejestry:
  Wtryskarka 1: 250.5 °C
  Wtryskarka 2: 249.9 °C
  Wtryskarka 3: 250.6 °C

```

Rysunek 11 Odczyt bieżących parametrów produkcji z bramy MateChain za pomocą protokołu Modbus TCP

Źródło: opracowanie własne

Brama poprawnie przetworzyła przychodzące ramki danych i zwróciła zmapowane wartości ze współdzielonej pamięci podręcznej (struktury HashMap). Podczas integracji z systemami SCADA ujawnił się problem charakterystyczny dla standardu Modbus TCP. Zgodnie z jego specyfikacją rejestry przyjmują wyłącznie 16-bitowe liczby całkowite, podczas gdy odczyty temperatury mają precyzję zmiennoprzecinkową (np. 250,5°C). Rozwiązano to, stosując skalowanie matematyczne w kodzie bramy: przed wysłaniem wartość jest mnożona przez 10 (250,5 kodowane jako liczba całkowita 2505), a po stronie klienta dzielona przez 10, co zachowuje wymaganą precyzję pomiarową. Test dowodzi, że zastosowanie łańcucha bloków nie zaburza wstecznej kompatybilności z klasycznymi systemami automatyki²⁷.

5.2.4. Wizualizacja danych w czasie rzeczywistym

Kolejnym elementem weryfikacji warstwy integracyjnej było przetestowanie autorskiego interfejsu graficznego. W trakcie symulacji procesów uruchomiono przeglądarkę i nawiązano połączenie z serwerem bramy pod adresem lokalnym na porcie 8090.

Panel operatorski załadował się poprawnie, prezentując przejrzysty widok parametrów telemetrycznych wszystkich aktywnych wtryskarek. Dane odświeżały się automatycznie na podstawie napływających asynchronicznie struktur JSON, bez ręcznego przeładowywania strony. System prawidłowo interpretował stany maszyn, oznaczając krytyczne anomalie temperaturowe.

²⁷ Modbus Organization, Inc., MODBUS Application Protocol Specification V1.1b3, [b.m.w.] 2012, s. 6, 12-16., dostęp online: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf [dostęp: 4.06.2026].



Rysunek 12 Interfejs graficzny WebUI prezentujący bieżące parametry pracy maszyn w czasie rzeczywistym

Źródło: opracowanie własne

Poprawne działanie panelu potwierdza, że system MateChain nie tylko archiwizuje dane historyczne z wykorzystaniem sieci blockchain, lecz także dostarcza ergonomicznych narzędzi nadzoru bieżącego, istotnych dla personelu obsługującego nowoczesną halę produkcyjną.

5.3. Ocena procedury audytu i odporności na sabotaż

Celem ostatniego etapu weryfikacji funkcjonalnej było przetestowanie narzędzia audytorskiego napisanego w języku Python. Zgodnie z założeniami projektowymi skrypt ma umożliwiać niezależnemu kontrolerowi pobranie, odszyfrowanie i matematyczną weryfikację paczek produkcyjnych z pominięciem lokalnej bazy danych przedsiębiorstwa.

Najpierw przeprowadzono test ścieżki prawidłowej. W konsoli uruchomiono skrypt z parametrami wejściowymi: identyfikatorem CID oraz oczekiwanym skrótem SHA-256, odczytanymi wcześniej z publicznego rejestru Solana. Aplikacja nawiązała połączenie z węzłem IPFS i pobrała zaszyfrowany plik binarny, po czym samodzielnie wyliczyła jego skrót kryptograficzny. Zgodność obu wartości pozwoliła przejść do deszyfracji AES-GCM i dekompresji GZIP. W terminalu zaprezentowano czytelną listę

zdekodowanych parametrów produkcyjnych, a audyt zakończył się komunikatem o sukcesie.

```

mati@mati-HP-EliteBook-725-G3:~/Pulpit/Praca_Dyplomowa/audytor$ python3 audytor.py --cid Qmd3LN8SH6b6HPkVuyGn6YG4G2MoD4uEYAY3fy5mS6grop --hash 08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404f

```

MATECHAIN – RAPORT AUDYTU PACZKI IPFS

Data audytu: 2026-06-02 15:23:30
CID paczki: Qmd3LN8SH6b6HPkVuyGn6YG4G2MoD4uEYAY3fy5mS6grop

Krok 1/5 – pobieranie paczki z IPFS

✓ Pobrano 868 bajtów

Krok 2/5 – weryfikacja integralności (SHA-256)

Oczekiwany SHA-256: 08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404f
Obliczony SHA-256: 08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404f
✓ Skrót się zgadzają – paczka jest autentyczna

Krok 3/5 – odszyfrowanie AES-256 GCM

Nonce (12 bajtów): 9dd4cc80a18368a21b2450ab
Tag autentyczności (16 b): a08abbd7d5c98b9564db874713d690cd
Ciphertext: 840 bajtów
✓ Odszyfrowano poprawnie – tag GCM zweryfikowany

Krok 4/5 – dekompresja gzip

Po dekompresji: 12579 bajtów (×15.0)

Krok 5/5 – parsowanie JSON Lines

✓ Sparsowano 88 rekordów

Analiza zawartości paczki (88 rekordów)

Liczba odczytów: 88
Liczba alarmów: 0 (0.0%)
Temperatura min: 249.85°C
Temperatura max: 250.59°C
Temperatura średnia: 250.19°C

Pierwsze 10 rekordów:

```

[2026-06-02 13:18:01] 2af1b5c5d7cf termopara_PT100_3 250.08°C
[2026-06-02 13:18:03] 2ff408844f18 termopara_PT100_1 249.85°C
[2026-06-02 13:18:04] a86d50eb01d9 termopara_PT100_2 250.27°C
[2026-06-02 13:18:06] 2af1b5c5d7cf termopara_PT100_3 250.12°C
[2026-06-02 13:18:08] 2ff408844f18 termopara_PT100_1 249.92°C
[2026-06-02 13:18:09] a86d50eb01d9 termopara_PT100_2 250.22°C
[2026-06-02 13:18:11] 2af1b5c5d7cf termopara_PT100_3 250.09°C
[2026-06-02 13:18:14] 2ff408844f18 termopara_PT100_1 249.92°C
[2026-06-02 13:18:15] a86d50eb01d9 termopara_PT100_2 250.18°C
[2026-06-02 13:18:15] 2af1b5c5d7cf termopara_PT100_3 250.14°C

```

... ostatnie 5 rekordów:

```

[2026-06-02 13:20:08] 2ff408844f18 termopara_PT100_1 250.06°C
[2026-06-02 13:20:09] 2af1b5c5d7cf termopara_PT100_3 250.21°C
[2026-06-02 13:20:10] a86d50eb01d9 termopara_PT100_2 250.58°C
[2026-06-02 13:20:12] 2ff408844f18 termopara_PT100_1 250.01°C
[2026-06-02 13:20:12] 2af1b5c5d7cf termopara_PT100_3 250.21°C

```

Zapis raportu

Zapis raportu

✓ Raport zapisany w raport_audytu.json

✓ AUDYT ZAKOŃCZONY POMYSŁNIE

```

mati@mati-HP-EliteBook-725-G3:~/Pulpit/Praca_Dyplomowa/audytor$

```

Rysunek 13 Prawidłowy przebieg audytu paczki IPFS zakończony sukcesem weryfikacji

Źródło: opracowanie własne

Drugi scenariusz miał wykazać odporność systemu na celowy sabotaż danych historycznych. Zasymulowano cyberatak polegający na nieautoryzowanej modyfikacji danych: przy uruchamianiu skryptu audytorskiego celowo podano błędny skrót kryptograficzny (symulując podmianę pliku w sieci IPFS) i ponownie zainicjowano procedurę weryfikacyjną.

Skrypt audytorski wykrył naruszenie integralności danych na wczesnym etapie analizy. Niezgodność skrótów wyzwoliła błąd autentyczności, co doprowadziło do natychmiastowego przerwania procesu, zablokowania deszyfracji pliku i wygenerowania czerwonego alertu o sfałszowaniu historii produkcyjnej.

```
mati@mati-HP-EliteBook-725-G3:~/Pulpit/Praca_Dyplomowa/audytor$ python3 audytor.py --cid Qmd3LN8SH6b6HPkVuyGn6YG4G2MoD4uEYAY3fy5mS6grop --hash 08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404d
```

```
=====
MATECHAIN – RAPORT AUDYTU PACZKI IPFS
=====
Data audytu:                2026-06-02 15:25:39
CID paczki:                  Qmd3LN8SH6b6HPkVuyGn6YG4G2MoD4uEYAY3fy5mS6grop
=====

Krok 1/5 – pobieranie paczki z IPFS
=====

✓ Pobrano 868 bajtów
=====

Krok 2/5 – weryfikacja integralności (SHA-256)
=====

Oczekiwany SHA-256:          08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404d
Obliczony SHA-256:           08912750f93bb6757e6d0c9254279e08a7dd255f494b2301bd8046c6120d404f
X Skrót się NIE zgadzają – paczka mogła zostać zmodyfikowana!
X PRZERWANO AUDYT – paczka mogła zostać sfałszowana
mati@mati-HP-EliteBook-725-G3:~/Pulpit/Praca_Dyplomowa/audytor$
```

Rysunek 14 Przerwanie procedury audytorskiej w wyniku wykrycia matematycznej niezgodności danych

Źródło: opracowanie własne

Wyniki obu eksperymentów potwierdzają skuteczność zaimplementowanej architektury kryptograficznej. Zastosowanie publicznego łańcucha bloków Solana jako niezależnego punktu odniesienia (kotwicy zaufania) pozwala wykrywać próby modyfikacji danych przechowywanych off-chain. Ponieważ wyliczony przez bramę skrót SHA-256 zostaje trwale zapisany w rozproszonym rejestrze za pomocą inteligentnego kontraktu, ukrycie awarii technologicznej lub sfałszowanie historii pracy maszyn przez personel

zakładu staje się niewykonalne. Procedura audytu realizuje tym samym założenia projektowe dotyczące niezaprzeczalności i integralności danych pomiarowych²⁸.

5.4. Analiza ekonomiczno-energetyczna

Wdrożenie mechanizmów kryptograficznych i technologii rozproszonego rejestru do klasycznych sieci automatyki przemysłowej wymaga oceny kosztów operacyjnych. Rozwiązanie informatyczne przeznaczone do komercjalizacji w przedsiębiorstwie produkcyjnym musi mieć uzasadnienie ekonomiczne oraz odpowiednią efektywność energetyczną.

5.4.1. Bilans kosztów transakcyjnych w sieci Solana

Głównym czynnikiem decydującym o kosztach utrzymania systemu MateChain są opłaty sieciowe pobierane przez walidatorów za zatwierdzanie transakcji on-chain. Sieć Solana wybrano ze względu na mechanizm konsensusu Proof of History, który znacząco obniża te koszty w porównaniu ze starszymi łańcuchami bloków.

Zgodnie z oficjalną specyfikacją protokołu bazowa opłata za standardową transakcję z pojedynczym podpisem wynosi 5000 lamportów, czyli 0,000005 natywnej kryptowaluty SOL. Aby oszacować całkowity miesięczny koszt utrzymania systemu, opracowano model matematyczny uwzględniający interwał archiwizacji logów w sieci IPFS oraz szacowaną liczbę anomalii krytycznych. Koszt ten można wyrazić następującym równaniem²⁹

$$C = \left(\frac{T}{t} + A \right) \times f \times P \quad (5.1)$$

²⁸ E. Barker - General, NIST Special Publication 800-57 Part 1 Rev. 5, 2020, s. 32 (definicja Trust Anchor).

²⁹ Transaction Fees | Solana, oficjalna dokumentacja sieci Solana, dostęp online: <https://solana.com/docs/core/fees> [dostęp: 4.06.2026].

C – całkowity koszt miesięczny,
 T – całkowity czas pracy maszyn w miesiącu (w sekundach),
 t – zaprogramowany interwał archiwizacyjny (120 sekund),
 A – szacowana miesięczna liczba nadzwyczajnych transakcji alarmowych,
 f – bazowa opłata transakcyjna (0,000005 SOL),
 P – uśredniony rynkowy kurs kryptowaluty Solana.

Przykład obliczeniowy: aby zobrazować rzeczywiste koszty operacyjne, przeprowadzono kalkulację dla pojedynczej wtryskarki pracującej w trybie ciągłym (24 godziny na dobę przez 30 dni). Całkowity czas pracy w miesiącu wynosi wówczas 2 592 000 sekund. Zgodnie z przyjętą architekturą system archiwizuje paczkę danych off-chain co 2 minuty. Zakładając wystąpienie 50 krytycznych anomalii technologicznych w skali miesiąca oraz przyjmując uśredniony kurs rynkowy kryptowaluty Solana na poziomie 245 PLN³⁰, całkowity miesięczny koszt transakcyjny wyniesie 26,52 PLN.

5.4.2. Efektywność energetyczna architektury

W dobie rosnącej świadomości ekologicznej oraz wprowadzania rygorystycznych norm środowiskowych dla zakładów produkcyjnych, kluczowym aspektem oceny systemu MateChain staje się jego bilans energetyczny. Klasyczne sieci łańcucha bloków pierwszej generacji, takie jak Bitcoin, opierają się na energochłonnym algorytmie dowodu pracy. Wymagają one potężnych nakładów obliczeniowych, co wiąże się z ogromnym śladem węglowym. Taka charakterystyka całkowicie dyskwalifikuje je z zastosowań w lekkich urządzeniach brzegowych, które tworzą zakładowy Internet Rzeczy.

Sieć Solana ogranicza ten problem. Dzięki hybrydowemu modelowi konsensusu łączącemu dowód stawki z historycznym dowodem czasu (Proof of Stake i Proof of History) nie wymaga ona kosztownego obliczeniowo procesu kopania bloków. Zgodnie z oficjalnymi raportami środowiskowymi fundacji Solana pojedyncza transakcja zużywa

³⁰ Kurs SOL/PLN według agregatora rynkowego CoinMarketCap, dostęp online: <https://coinmarketcap.com/currencies/solana/sol/pln> [dostęp: 13.06.2026].

średnio ułamek wata energii czyli wartość porównywalną z kosztem energetycznym kilku zapytań w standardowej wyszukiwarce internetowej³¹.

Również sama brama sieciowa, realizująca procesy kryptograficzne po stronie zładu, została zoptymalizowana już na poziomie wyboru technologii. Zastosowano język Rust, który nie korzysta z automatycznego mechanizmu odświeżania pamięci i pozwala oszczędnie zarządzać zasobami sprzętowymi. Dzięki temu węzeł MateChain można uruchomić na energooszczędnych komputerach przemysłowych o niskim poborze mocy. System pokazuje zatem, że decentralizację i bezpieczeństwo danych można realizować bez negatywnego wpływu na bilans energetyczny nowoczesnych, zrównoważonych fabryk.

5.5. Ograniczenia i słabe strony zaproponowanego rozwiązania

Choć system MateChain skutecznie realizuje paradygmat niezaprzeczalności logów produkcyjnych, jak każde rozwiązanie inżynierskie ma ograniczenia wynikające ze specyfiki zastosowanych technologii zdecentralizowanych. Krytyczna ocena projektu pozwala wskazać obszary wymagające dalszej optymalizacji przed ewentualną komercjalizacją.

Pierwszym zidentyfikowanym problemem jest trwałość danych archiwizowanych w sieci IPFS. W odróżnieniu od tradycyjnych serwerów chmurowych protokół IPFS nie gwarantuje bezwarunkowej i trwałej dostępności plików. Węzły sieci okresowo uruchamiają proces odświeżania pamięci, usuwając nieużywane dane, przez co bez odpowiednich mechanizmów utrwalania zarchiwizowane logi mogłyby z czasem stać się niedostępne. Aby temu zapobiec, konieczne jest aktywne „przypinanie” (pinning) paczek na własnym węźle lub korzystanie z usług zewnętrznych dostawców. Brak natywnych mechanizmów rynkowych w rdzeniu protokołu zmusza do stosowania takich płatnych usług, co podnosi stałe koszty operacyjne systemu³².

³¹ *Solana Energy Use Report*, oficjalna dokumentacja fundacji Solana, dostęp online: <https://solana.com/environment> [dostęp: 4.06.2026].

³² J. Benet, dz. cyt.

Kolejną słabą stroną są opóźnienia sieciowe podczas procedury audytorskiej. Zdecentralizowana architektura IPFS, oparta na odpytywaniu rozproszonych tablic mieszających (DHT), wprowadza zauważalne narzuty czasowe. Z przeprowadzonych badań wynika, że lokalizowanie dostawców danych w publicznej sieci bywa niestabilne, a czas pobrania paczki po identyfikatorze CID może w skrajnych przypadkach sięgać kilkudziesięciu sekund co oznacza wynik znacznie gorszy niż w zoptymalizowanych, scentralizowanych protokołach HTTP³³.

Istotną barierą wdrożeniową jest też czynnik organizacyjny związany z wykorzystaniem łańcucha bloków. Choć koszty transakcji w publicznej sieci Solana są ułamkowe, system do ciągłego działania wymaga utrzymywania dodatniego salda kryptowaluty na portfelu bramy brzegowej. Dla wielu tradycyjnych przedsiębiorstw konieczność cyklicznego zakupu aktywów wirtualnych o silnie zmiennej wartości może stanowić trudną do zaakceptowania barierę księgową. Ponadto poprawne funkcjonowanie systemu wymusza otwarcie izolowanej dotąd sieci automatyki operacyjnej na komunikację z publicznym Internetem. Takie zatarcie fizycznych granic sieci firmowej jest charakterystyczne dla rozwiązań mgły obliczeniowej, ale zwiększa powierzchnię ataku i wymusza dodatkową logikę zabezpieczeń, co może budzić opór zakładowych zespołów cyberbezpieczeństwa³⁴.

³³ Tamże.

³⁴ R. Siudak, dz. cyt., s. 1.

ZAKOŃCZENIE

Niniejsza praca inżynierska dotyczyła wykorzystania technologii blockchain do zabezpieczania danych w automatyce przemysłowej. Głównym celem było zaprojektowanie i uruchomienie systemu, który potrafi wykrywać i trwale zapisywać ważne zdarzenia z maszyn produkcyjnych.

Cel pracy został osiągnięty. Zbudowano system MateChain, który na bieżąco pobiera dane z maszyny, a następnie je archiwizuje. Testy pokazały, że wszystkie bieżące odczyty oraz nietypowe awarie są poprawnie rejestrowane, zabezpieczane kryptograficznie i zapisywane w sieci. System działa stabilnie i skutecznie blokuje próby zmiany zapisanych logów przez osoby nieuprawnione.

Analiza kosztów potwierdziła, że projekt jest opłacalny. Obliczenia dla pojedynczej wtryskarki w ujęciu miesięcznym wykazały, że koszt ochrony danych to niecałe 27 złotych. To dowód, że system można wdrożyć w prawdziwych zakładach bez dużego obciążania budżetu. Tak niski koszt uzyskano dzięki zastosowaniu modelu hybrydowego. Zapisywanie dużych plików bezpośrednio w łańcuchu bloków byłoby zbyt drogie i obciążałoby system. Dlatego jako zewnętrzny magazyn danych wykorzystano sieć IPFS, a w samej sieci Solana zapisywane są tylko unikalne skróty (hasze). To rozwiązanie obniżyło koszty i pozwoliło zachować pełną historię pracy maszyny.

Zbudowany system jest uniwersalny i można go dalej rozwijać. W przyszłości warto dodać moduł powiadomień, który informowałby pracowników utrzymania ruchu o wystąpieniu awarii. Ponieważ system obsługuje już najpopularniejszy protokół Modbus TCP, kolejnym krokiem powinno być dodanie standardów takich jak OPC UA czy PROFINET. Ułatwiłoby to podłączenie bramki do maszyn różnych producentów. Elastyczność tego rozwiązania pozwala również na jego użycie w innych branżach, na przykład w farmacji lub przemyśle spożywczym, gdzie dokładne raportowanie jakości jest kluczowe.

BIBLIOGRAFIA

LITERATURA

- Andreeva O., Gordeychik S., Gritsai G. i in., *Industrial Control Systems Vulnerabilities Statistics*, Kaspersky Lab, [b.m.w.] 2015.
- Benet J., *IPFS – Content Addressed, Versioned, P2P File System*, [b.m.w.] 2014.
- Klabnik S., Nichols C., *Programowanie w języku Rust. Oficjalny podręcznik. Wydanie II*, Gliwice 2023.
- Mychlewicz C., Piątek Z. (red.), *Od Industry 4.0 do Smart Factory. Poradnik menedżera i inżyniera*, Warszawa 2017.
- Siudak R., *Cyberbezpieczeństwo przemysłu – polskie szanse, inwestycje, innowacje*, „Brief Programowy” 2020, nr lipiec.
- Yakovenko A., *Solana: A new architecture for a high performance blockchain v0.8.13*, [b.m.w.] 2018.

DOKUMENTY NORMALIZACYJNE I SPECYFIKACJE TECHNICZNE

- Advanced Encryption Standard (AES)*, FIPS PUB 197, National Institute of Standards and Technology, Gaithersburg 2001.
- Barker E., *Recommendation for Key Management. NIST Special Publication 800-57 Part 1 Rev. 5*, National Institute of Standards and Technology, Gaithersburg 2020.
- Modbus Organization, Inc., *MODBUS Application Protocol Specification V1.1b3*, [b.m.w.] 2012.
- Secure Hash Standard (SHS)*, FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg 2015.
- Standard pisania prac inżynierskich*, Akademia WSB, Dąbrowa Górnicza [b.r.w.].

NETOGRAFIA

- www.anchor-lang.com [dostęp: 4.06.2026 r.]
- coinmarketcap.com/currencies/solana/sol/pln [dostęp: 13.06.2026 r.]
- docs.ipfs.tech/concepts/how-ipfs-works [dostęp: 4.06.2026 r.]
- modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf [dostęp: 4.06.2026 r.]
- solana.com/docs/core/accounts [dostęp: 4.06.2026 r.]
- solana.com/docs/core/fees [dostęp: 4.06.2026 r.]
- solana.com/docs/core/idl [dostęp: 4.06.2026 r.]
- solana.com/environment [dostęp: 4.06.2026 r.]
- www.sqlite.org/wal.html [dostęp: 4.06.2026 r.]

WYKAZ TABEL

Tabela 1 Konfiguracja interfejsów sieciowych i mapowania portów środowiska testowego	41
--	----

WYKAZ RYSUNKÓW

Rysunek 1 Trójwarstwowa architektura systemu	18
Rysunek 2 Schemat blokowy dwutorowej strategii rejestracji zdarzeń	20
Rysunek 3 Schemat procedury audytorskiej i weryfikacji kryptograficznej.....	22
Rysunek 4 Schemat działania pętli głównej symulatora urządzeń brzegowych	26
Rysunek 5 Schemat blokowy logiki klasyfikacji zdarzeń i dekapulacji danych w bramie sieciowej	31
Rysunek 6 Schemat rozkładu pamięci i alokacji struktury danych konta BatchData w programie Solana	34
Rysunek 7 Schemat przepływu danych i architektury routingu serwera panelu operatorskiego.....	39
Rysunek 8 Wykrycie anomalii temperaturowej oraz wygenerowanie sygnatury transakcji w terminalu bramy	42
Rysunek 9 Zarejestrowany alarm krytyczny w publicznym eksploratorze sieci Solana (klaster devnet)	43
Rysunek 10 Proces agregacji i wysyłki paczki historycznej do systemu IPFS	44
Rysunek 11 Odczyt bieżących parametrów produkcji z bramy MateChain za pomocą protokołu Modbus TCP	45
Rysunek 12 Interfejs graficzny WebUI prezentujący bieżące parametry pracy maszyn w czasie rzeczywistym	46
Rysunek 13 Prawidłowy przebieg audytu paczki IPFS zakończony sukcesem weryfikacji	48
Rysunek 14 Przerwanie procedury audytorskiej w wyniku wykrycia matematycznej niezgodności danych.....	49

